Highlights

Which Design Choices in HTTP Route Matching Matter, and When

Alex I. Tsvetanov

- Route cost depends on table size, working set and parameters.
- Literal methods use a hash; other methods use a segment trie.
- The matcher loses to gin on a small wild table.
- Compile-time checks reject malformed routes at compilation.
- End-to-end latency against external routers is unmeasured.

Which Design Choices in HTTP Route Matching Matter, and When

Alex I. Tsvetanov[1,*]

Faculty of Computer Systems and Technologies, Technical University of Sofia, ul. Professor Georgi Bradistilov 11, Sofia, Bulgaria

Abstract

Routing cost depends on table size, the working set that is queried, and whether routes capture parameters. This paper asks which design choices in route matching matter, and when, and it reports the losses. It measures the public RegexMatcher after its DFA construction, with the route target included. The arms are a radix trie, a sequential scan, a static-path hash, and pinned routers in C, C++, Rust and Go. Each external router runs in the best configuration its own documentation states. The setting is one core of an AMD Ryzen 7 5800H, with Clang 22.1.8, and every arm built for that processor. v2 was not slower than the fastest competitor in 34 of 35 cells, and faster in 33. In those faster cells it took 0.1691 to 0.9053 of that competitor's time. At the wild shape with 10 routes, neither claim holds against gin. v2 ran more instructions than gin in 2 cells. It built large tables more slowly than the median competitor in 4 cells. A minimal server answered more requests per second with v2 than with v1 in all 9 cells. That comparison is inside this paper's server and says nothing about the external routers. End-to-end latency against those routers is absent from this manuscript. A compile-time table passed its per-cell test in 20 of 21 cells, so that hypothesis does not hold.

Keywords: HTTP routing, route matching, trie, compile-time checking, benchmarking, C++, non-inferiority

1. Introduction

An HTTP server must decide which code answers each request. Its router takes the request's method and path and returns the handler of the route that matches, with the values that the route's parameters capture. The lookup runs once per request, beside parsing and input and output. A server that answers many small requests on few cores pays for it on each one.

Web frameworks in C, C++, Rust and Go each ship a router of their own, and the routers differ in structure. Some compress the routes into a radix tree Schmidt (2019); Gin contributors (2026); go-chi contributors (2026); Ahmed (2026); viz-rs contributors (2025). Others build a trie of whole path segments Pistache contributors (2024); uNetworking contributors (2026); Glaze contributors (2026) or a prefix trie that uses regular expressions for patterns r3 contributors (2026). Crow builds a character trie and merges chains of single children CrowCpp contributors (2026a). Still others try a list of patterns in order Oat++ contributors (2024); cpp-httplib contributors (2026); Actix contributors (2026), or match a regular expression per parameter route Drogon contributors (2026). They also differ in what a route means: which of two matching routes wins, whether a trailing slash matters, and what a catch-all captures. The peer-reviewed literature on router speed is thin. We found two papers on routing with tries Zhang and Xu (2015); Dsouza et al. (2018). We also found benchmark suites kept beside routers Schmidt (2020); Ahmed (2026); SwornSystems contributors (2026); vm-001 (2024).

The author's earlier, co-authored work introduced RegexMatcher Stankov and Tsvetanov (2024). It matches one string against many regular expressions at once, and web servers whose routes are regular expressions

*Corresponding author.

Email address: atsvetanov@tu-sofia.bg (Alex I. Tsvetanov)

[1]ORCID 0009-0001-8369-150X.

motivated it. A route written as a regular expression loses what a router needs, however. Precedence then follows the order of registration, a typed parameter is only a character class, and every lookup runs an automaton over the whole path. This paper reworks RegexMatcher into a route matcher, RegexMatcher v2, and keeps the regular-expression engine, v1, beside it in the same library.

The paper makes four contributions:

- A survey of the route matchers of sixteen routers written in C, C++, Rust and Go. It states how each one stores its routes and what its lookup returns. It then measures all of them on the same tables, with every answer checked, and runs eight probes of the path rules on each.
- RegexMatcher v2, a route matcher in C++23. A method whose routes are all literal gets an exact-match table, and every other method a trie of path segments. The tables live in flat arrays and a lookup never allocates. Patterns, handler signatures and duplicate routes are checked when the program compiles.
- A comparison under seven hypotheses that were pre-specified before the run, in a file of the paper's repository (Section 4.1). It covers lookup time, work per lookup, allocation, build time and memory, the compile-time checks, and a table built at compile time. Its server compares v2 with v1 only.
- The results as the pre-specified rules decide them, losses included.

The question is which design choices matter, and when, losses included. v2 was not slower than the fastest competitor in 34 of 35 cells, and faster in 33. Every such claim holds for one measured setting only: one core of an AMD Ryzen 7 5800H (Zen 3) and Clang 22.1.8. Every arm is compiled for that processor's own instruction set, and the competitors run in the configurations of Appendix Appendix B. The compile-time checks come second.

Section 2 surveys related work and the competing routers. Section 3 describes v2. Section 4 gives the pre-specified design of the measurement. Section 5 reports each hypothesis and the exploratory runs. Section 6 discusses the losses. Section 7 states the threats to validity. Section 8 concludes.

2. Related Work and a Survey of Route Matchers

2.1. Tries and Lookup Structures

A route table is a set of keys made of path segments, so the trie is its natural structure. De la Briandais and Fredkin described the digital search tree, which Fredkin named "trie" De La Briandais (1959); Fredkin (1960). PATRICIA compresses a binary trie by skipping nodes with a single branch Morrison (1968). The radix trees of several routers follow this line. The double-array trie encodes a trie in flat arrays Aoe (1989). The burst trie mixes trie nodes with other containers Heinz et al. (2002). The adaptive radix tree chooses a node's layout by its number of children, and its authors report lookups comparable to hash tables Leis et al. (2013). v2 borrows two of these ideas. It flattens its trie into arrays after the build, without the double-array encoding. It also changes how a node finds a literal child once the node has many children (Section 3).

Address lookup in routers of the network layer is a large neighbouring field. Its surveys cover the design space Ruiz-Sanchez et al. (2001); Gupta and McKeown (2001); Taylor (2005). Its methods include adaptive branching Andersson and Nilsson (1993), level compression Nilsson and Karlsson (1999), prefix expansion Srinivasan and Varghese (1999) and tables small enough for a cache Degermark et al. (1997). Others are binary search over hash tables Waldvogel et al. (1997), the tree bitmap Eatherton et al. (2004) and Poptrie Asai and Ohara (2015). Poptrie's authors evaluate their structure with an analysis of processor cycles. We follow that practice and report hardware counters beside every timing. Name lookup in named data networking is the closest analogue, because names there are hierarchies of components Wang et al. (2012); So et al. (2013); Yuan and Crowley (2015). Those structures find the longest matching prefix. A route matcher instead needs an exact match with typed parameters, a catch-all, precedence and captured values.

v2's exact-match table and its hashed trie nodes use open addressing Peterson (1957). Robin Hood hashing Celis et al. (1985), cuckoo hashing Pagh and Rodler (2004), and perfect and minimal perfect hashing Fredman et al. (1984); Czech et al. (1992); Belazzougui et al. (2009); Pibiri and Trani (2021) are alternatives. GNU gperf generates perfect hash functions as source code Free Software Foundation (2025), and the frozen library offers constant-evaluated hash containers Serge sans Paille (2024). A table that is built when the program compiles knows its keys, so a perfect hash would be possible there. v2 builds the same layout at compile time and at run time instead, so that one lookup serves both. We did not test a perfect hash, and we make no claim about it.

2.2. Research on HTTP Routing

Zhang and Xu replace a table of regular expressions with a trie searched depth first, with static segments in a hash table Zhang and Xu (2015). A work-in-progress paper embeds a hash trie for REST URLs in a server Dsouza et al. (2018). Zeng and colleagues match literal URLs from a fixed start in HTTP streams, without parameters or precedence Zeng et al. (2017). The Go benchmark suite of Schmidt compares Go routers on the route sets of real APIs Schmidt (2020). Our exploratory real-world table is its GitHub API list. That suite times Go routers inside Go. matchit's documentation ranks Rust routers by a benchmark Ahmed (2026), and wayfind keeps a suite that times Rust routers SwornSystems contributors (2026). A gateway benchmark times API gateways on the route sets of real interfaces vm-001 (2024). Each times routers of one language or one kind. We time routers of four languages behind one harness, with null arms and checked answers. The path grammar that a parameter accepts comes from RFC 3986 Berners-Lee et al. (2005). The meaning of HEAD and of a 405 answer comes from RFC 9110 Fielding et al. (2022).

2.3. The Routers Measured

Table 1 lists the sixteen competitors. Each was the latest release of its project when the hypotheses were frozen, or, for r3, the head of its maintained branch. The table says how each stores its routes, as its source or documentation at that version states it. It also says what its lookup returns. Appendix Appendix A gives the exact call that the harness times for each.

Two of these are libraries that larger frameworks build on: matchit, which axum uses, and actix-router, the router of actix-web. axum's latest release, 0.8.9, pins matchit 0.8.4 Tokio contributors (2026), so the arm measures matchit's latest release rather than the one that axum uses. gin's tree derives from httprouter's and carries its notice. Drogon's router needs a running application, so its lookup is transcribed into the harness from its source at the pinned tag. The transcription is a threat to validity (Section 7).

The routers disagree on what a route means. httprouter refuses a literal and a parameter at the same position for one method. Go's ServeMux panics at registration when two patterns overlap and neither is more specific. matchit refuses ambiguous routes when they are inserted. Lists tried in order give precedence to the earlier route. Section 5 runs eight probes of the path rules on every arm and shows where the measured routers differ.

2.4. Routes Checked When the Program Compiles

Typed web frameworks check routes against handlers in several languages. In Servant, a web API is a Haskell type, interpreted as a server, a client and documentation Mestanogullari et al. (2015). Ocsigen uses OCaml's types for valid links and service calls Balat (2006). Yesod turns a route language into Haskell code with Template Haskell. Its route quasiquoter stops the compilation when a route line is malformed or two routes may overlap. Each route's arguments take the types of its dynamic pieces Yesod Web Framework (2026); Yesod contributors (2025). The mechanism is quasiquotation, which checks a language written as a string when the program compiles Mainland (2007). Play compiles its route file but resolves overlaps by declaration order Play Framework (2026). Rocket requires each dynamic parameter to appear as a typed argument of its handler. Colliding routes stop Rocket at launch, not at compile time Rocket contributors (2024). Beyond routing, Ur/Web is a statically typed language for web applications Chlipala (2015).

Table 1: The sixteen routers compared with v2. "Structure" is read from the source or the documentation at the version measured. "Values" is what the lookup returns for a parameter. Appendix Appendix B gives the build of each.

Router	Lang.	Version	Structure	Values
Crow CrowCpp contributors (2026a)	C++	v1.3.4	a trie; each node holds a key, a parameter type and a vector of children	copies into strings
Drogon Drogon contributors (2026)	C++	v1.9.13	a lower-cased copy of the path in two hash maps, then a regular expression per parameter route, in turn	a new vector of strings
Oat++ Oat++ contributors (2024)	C++	1.3.1	a list of patterns, tried in order	views; the tail copied
Pistache Pistache contributors (2024)	C++	v0.4.26	a tree of segments whose children sit in hash maps	copies into strings
Boost.URL Boost.URL contributors (2026)	C++	1.92.0	the example router of the library: segment templates, literal first	views
r3 r3 contributors (2026)	C	2.0 branch, 83d362f	a prefix trie compiled from the paths, with PCRE2 10.49	views
uWebSockets uNetworking contributors (2026)	C++	v20.80.0	a tree of segments under the method; children sorted literal, parameter, catch-all	views; none for a catch-all
glaze Glaze contributors (2026)	C++	v9.0.0	a hash map of static paths, then a trie with hashed literal children	decoded strings in a map
cpp-httplib cpp-httplib contributors (2026)	C++	v0.58.0	a list per method, tried in order; a regular expression where a pattern needs one	copies, or views of the expression's groups
matchit Ahmed (2026)	Rust	0.9.2	a radix trie; a literal before a parameter	offsets
actix-router Actix contributors (2026)	Rust	0.5.4	a list of resource definitions, tried in order	offsets
path-tree viz-rs contributors (2025)	Rust	0.8.3	a compressed prefix tree with literal and parameter children	offsets
httprouter Schmidt (2019)	Go	v1.3.0	a compressing radix tree; a literal and a parameter never share a segment	offsets
gin Gin contributors (2026)	Go	v1.12.0	httprouter's tree, extended to let a literal stand beside a parameter	offsets
net/http The Go Authors (2026); Amsterdam (2024)	Go	Go 1.27.1	the ServeMux; the most specific pattern wins	copies
chi go-chi contributors (2026)	Go	v5.3.2	a radix tree with literal, regular-expression, parameter and catch-all nodes	offsets

Two C++ routers among the related work check part of a route when the program compiles. Crow's route macro derives the argument types from the parameter tags of a pattern, and a handler that cannot take them does not compile. A pattern without a leading slash, and a route defined twice, are found at run time CrowCpp contributors (2026b). RESTinio builds a route from parsers whose results reach the handler as separate arguments, so a mismatched handler does not compile. Its route is a C++ expression rather than a pattern string Stiffstream (2025).

v2's front end builds on the C++ techniques that check a string argument when the program compiles. The control string of printf decides the types of the arguments that follow it, and Danvy typed printf in ML by changing that string's representation Danvy (1998). Before strings could be constant expressions, Porkoláb and Sinkovics built a parser generator from template metaprograms, with a type-safe printf as its demonstration Porkoláb and Sinkovics (2010). Class types as template parameters let a string literal be a template argument Snyder and Dionne (2018). Compile-time regular expressions pass a pattern as such an argument and check it when the program compiles Dusíková (2019, 2026). The standard formatting library makes an invalid format string ill-formed Zverovich (2021); Zverovich et al. (2026). A Scala library checks regular expressions during type checking, with match types Blanvillain (2022). Constant evaluation itself has grown from generalised constant expressions Dos Reis and Stroustrup (2010); Stroustrup (2020). The standard leaves the limits of constant evaluation to the implementation Smith et al. (2020). Clang and MSVC expose them as a number of steps, set with `-fconstexpr-steps` and `/constexpr:steps` The Clang Team (2026); Microsoft (2026). A table built at compile time should cost nothing when the program runs, which is C++'s aim of zero-overhead abstraction Stroustrup (2012). H7 tests that aim on v2's tables.

2.5. Measurement

Small changes to an experimental setup can bias a performance evaluation Mytkowicz et al. (2009). Benchmarks need repetition at several levels and honest estimates of variation Georges et al. (2007); Kalibera and Jones (2013); Hoefler and Belli (2015); Curtsinger and Berger (2013); Laaber et al. (2019). Counts that should be exact, such as retired instructions, vary from run to run Weaver et al. (2013), so we report counters as medians over processes. The harness reads counters through the Linux interface Linux man-pages project (2026); Browne et al. (2000) and times with nanobench Leitner-Ankerl (2026). The intervals are bias-corrected and accelerated bootstrap intervals Efron (1987); Efron and Tibshirani (1993); DiCiccio and Efron (1996), cross-checked against SciPy's Virtanen et al. (2020). The family of cells is controlled with Holm's step-down procedure Holm (1979). The tests against a margin follow the framework of non-inferiority and equivalence testing Schuirmann (1987); Wellek (2010).

2.6. What This Paper Adds

Typed frameworks check routes when the program compiles, and C++ libraries check strings the same way. The sources we read report no comparison of lookup time against radix-tree routers, for typed frameworks or for C++ string checks. Fast routers document radix trees that match without garbage or copies Schmidt (2019); Ahmed (2026). None of the C++ sources we read checks a route's pattern, its handler's signature and its duplicates together when the program compiles. Yesod does all three in Haskell, and reports no comparison of lookup time. The router evaluations we found are benchmark suites kept beside routers, of Go routers, of Rust routers and of API gateways on real route sets Schmidt (2020); Ahmed (2026); SwornSystems contributors (2026); vm-001 (2024). One more compares a router with one table-based router Zhang and Xu (2015). We found none that measures routers of several languages on the same tables, checks their answers, and decides under pre-specified tests. This paper does all three for one C++ route matcher. We do not claim that v2 is the first typed router, or the first segment trie with hashed children. Pistache's tree and the router of Zhang and Xu are both segment structures with hashed literal children.

3. RegexMatcher v2

3.1. From a Set of Regular Expressions to a Route Matcher

RegexMatcher v1 matches one string against a set of regular expressions in a single pass over a prefix tree Stankov and Tsvetanov (2024, 2026). The paper measures its engine at commit `d16f30a8`. To route with it, a wrapper turns each pattern into a regular expression with one capture group per parameter, and compiles one automaton per method. The lowest route number among the matches wins, so precedence follows registration order. The wrapper finds a 405 by trying the other methods.

v2 adds a route matcher to the same library and leaves v1's engine unchanged. The route matcher is a second set of headers in its own namespace, `matcher::route`, with its own build target. It needs C++23 and the standard library only, and neither part of the library includes the other. Its parser, table, lookup and compile-time table began as a pre-release that a development round measured (Section 4.1). The paper measures v2 at commit `d1d73e99b`.

3.2. Patterns and Their Meaning

A pattern starts with a slash and splits at every slash into segments (Table 2). A literal segment may hold only the bytes that RFC 3986 allows in a path segment, the pchar set Berners-Lee et al. (2005). A pattern holds at most 32 segments and 16 parameters, and a parameter name appears once.

Table 2: The segments of a pattern.

Segment	**Matches**
`users`	the same bytes
`{name}` or `{name:string}`	one non-empty segment
`{name:u64}`	one segment that is an unsigned decimal number of type u64
`{name:i64}`	one segment that is a signed decimal number of type i64
`{*name}`	the non-empty rest of the path; only as the last segment

Seven rules give a route its meaning. They are also the rules against which every arm's answers are checked (Section 4.6).

1. Paths are compared and captured as sent. Nothing is percent-decoded.
2. A parameter matches one segment of one or more bytes, so it accepts every pchar.
3. A catch-all matches a non-empty remainder of the path, slashes included, as sent.
4. A trailing slash is strict. The matcher never redirects.
5. At every segment, from left to right, a literal beats a typed parameter, which beats a plain parameter, which beats a catch-all. The most specific route wins in any order of registration.
6. The matcher first looks for the request method's most specific route. Without one, it answers "method not allowed" with the set of methods whose routes match, or "not found".
7. Two routes of one method that match exactly the same paths are an error. A compile-time table fails to compile, and a build at run time returns an error.

What HEAD and OPTIONS mean is the server's choice. The matcher answers for the method it is asked.

3.3. The Table

One table holds the routes of every method. A method whose routes are all literal is answered by an exact-match table alone. It uses open addressing over a hash of the whole path and the method. A slot holds the hash, the method, the route and the place of the path's bytes in a shared arena, and a hit is confirmed by comparing those bytes.

Any other method gets a trie over path segments. Its fully literal routes stay in the trie too, because the walk tries literal children first and so finds them first. A node records where its literal children start and how many there are. It also records its typed, plain and catch-all children, the route that ends at it, and how many kinds of child it has. An edge to a literal child holds the first 8 bytes of the label as one machine word, the child, and the place of the whole label in the arena. A node with up to 8 literal children scans them in order. A wider node keeps them in a power-of-two array of slots, hashed by the word and the segment.

The four arrays of a table are the nodes, the edges, the exact-match slots and the arena. After the build they lie in one block aligned to a page of 4,096 bytes. Each array starts at the offset that a table built at compile time gives it. So a table has the same layout in both forms, and only its place in memory differs.

3.4. The Lookup

A lookup takes the table, the method and the path, and fills a result in place. The result holds the status, the route, and up to 16 captured values. Each value is a pointer and a length into the request's path, so a lookup never allocates. On "method not allowed" the result also holds the set of methods that match.

For a method whose routes are all literal, the lookup probes the exact-match table. Otherwise it walks the method's trie one segment at a time. It finds the end of a segment by loading the path a word at a time and searching the word for a slash, without reading outside the path. It then tries the literal children by word and length, comparing the remaining bytes only for labels longer than a word. Then it tests the typed children, then the plain parameter, then the catch-all. A node with more than one kind of child leaves a choice point. If the walk fails deeper down, it returns to the last choice and tries the next kind. A node with one kind of child never causes a return. Without a route of the request's method, the lookup tries each other method that has routes and reports the set that matches.

3.5. Building a Table at Run Time

The build takes a span of route specifications, each a method, a pattern and a route number. It returns the table, or an error that names the failing route. The build parses every pattern into one shared array of segment records, which holds only the segments that each pattern has. A route's entry is then small: its method, route, specification, first segment and segment count. The build sorts indices of entries rather than the entries themselves, and splits them between the exact-match table and the tries. One scratch stack holds the literal children of the node under construction. The arrays are reserved from counts known after the sort, and the result is copied once into the page-aligned block.

3.6. Checked When the Program Compiles

The checked front end passes a pattern as a template argument, a string literal held in a class type Snyder and Dionne (2018). One static assertion per kind of error then stops the compilation with a message that names the error, such as `route pattern: unclosed '{'`. The front end derives the C++ type of each parameter: `std::string_view`, `std::uint64_t` or `std::int64_t`. A route declaration pairs a method and a pattern with a handler. The handler must take one argument per parameter, in order, with those types, and then the server's context arguments. Two static assertions check the count and then the types. A generic lambda has no fixed arity, so it is checked by whether it can be called with those arguments.

A table of declarations is built during constant evaluation by the same build function as at run time. A malformed pattern or two routes of one method that match the same paths stop the compilation. The compiler

Listing 1: The checked front end. Each commented declaration at the end, added to the array, stops the compilation with the reason shown.

```
#include <matcher/route.hpp>
using namespace matcher::route;

struct Ctx { /* the server's request and response */ };
std::string get_user(std::uint64_t id, Ctx& c);
std::string get_file(std::string_view name, std::string_view rest, Ctx& c);

using H = handlers<std::string, Ctx&>;
inline constexpr H::decl kApi[] = {
    H::get<"/users/{id:u64}", &get_user>(),
    H::get<"/files/{name}/{*rest}", &get_file>(),
    H::post<"/users/{id:u64}", &get_user>(),
};
inline constexpr auto kTable = make_route_table<kApi>();
static_assert(kTable.table.find(0u, "/files/a/b/c").route == 1);  // 0u is GET

// H::get<"/u/{id", &get_user>()        route pattern: unclosed '{'
// H::get<"/u/{id:u64}", &get_file>()    route handler: takes a different number
//                                         of parameters than the route pattern has
// H::get<"/users/{n:u64}", &get_user>() compile_time_table_has_duplicate_routes
```

then names a function that tells the reason. Listing 1 shows the front end. The negative-compilation cases of H5(a) cover each of these errors.

A table built at compile time costs constant-evaluation steps and compiler memory. Clang and MSVC bound the steps per evaluation The Clang Team (2026); Microsoft (2026). The harness compiles each table with a budget of 33,554,432 steps. Section 5.9 reports what each table needed.

3.7. What the Rework Changed

Before the run, the engineering of v2 had three targets, each with a rule of adoption declared before anything was measured. The first was the build time of large tables, which the development round had found slower than the median competitor's. The rework of Section 3.5 met its rule, a lower build time in every development cell, and was adopted. It left the table's contents identical, byte for byte, to those of the pre-release.

The second target was the instructions per lookup at `mixed-disjoint` with 10 routes, where gin had run fewer. A node of gin's tree compares its whole prefix at once, which can span several segments, while v2 takes one trie step per segment. The development round took this for the cause. Chained edges, and two builds of a separate chain node, were built and measured on the lab host. Each either raised the instructions of other cells or did not beat gin's count, which its rule forbade, so none was adopted. The loss was therefore expected, and H2's text predicted it before the run.

The third target was the gap between a table built at compile time and one built at run time. Both arms of the harness now share one adapter, and both tables share one layout (Section 3.3).

None of this changed the lookup's code. The out-of-line lookup that every v2 arm calls compiles to identical machine code from the pre-release and from v2, with the run's compiler and flags. The ablation arm of Section 4.3 keeps the pre-release's build and layout, so it measures what the rework changed.

4. Methods

4.1. Pre-Specification and the Development Round

The hypotheses, the design of the runs, the statistics and the rules of decision were pre-specified in one file of the paper's repository, `hypotheses-round2.md`. Its status line records that it was frozen before the run. From then on, every change went into its revision log, dated and with its reason. Section 4.13 describes the log's entries. The repository's public history begins after the run. It holds the file and its log, but by itself it cannot show when the file was frozen. The frozen text names the analysis that computes the decisions, and states that the analysis's tests passed on synthetic runs. Those tests are in the repository. The two files of macros that carry its numbers into this paper were extended after the run, from its outputs and raw data, without changing a decision.

A development round measured a pre-release of the same lookup code, with the same harness and design, on other seeds. It guided the changes that Section 3.7 describes. Its numbers are not reported, and none of its seeds was reused. The seeds of the run had timed nothing before it. They had served only to check answers, in the agreement tests and the sanitizer records.

4.2. Host and Builds

Every measurement ran on one lab host, L: an AMD Ryzen 7 5800H (Zen 3) with Linux 7.2.3. The pre-specification froze the host, with no package update and no restart from the first sanitizer record to the end of the last run. Every process ran on CPU 2, whose sibling hardware thread, CPU 3, ran nothing of ours. The run recorded the performance governor, boost off, a mean clock of 3,179 MHz, transparent huge pages always on, and the NMI watchdog on.

C and C++ were compiled by clang 22.1.8, Rust by rustc 1.98.1, and Go by Go 1.27.1. Every arm was compiled for L's own instruction set. The C and C++ arms used `-O3 -DNDEBUG -std=c++23 -march=native`, the Rust arms `-C target-cpu=native` and the Go arms `GOAMD64=v3`. On L these resolve to the target znver3. Two competitors point to a native build. glaze's performance guide, written for its serialization, recommends a native build when a program runs where it was built, and uWebSockets' own example build uses one. Fairness needs the same level for every arm, v2 included. It also keeps H2's subtraction matched, because the harness's loop is compiled into each arm's translation unit. Each build writes the resolved target into a header that every target compiles. A build for another instruction set therefore has other inputs, and the gate of Section 4.7 refuses it.

Link-time optimization is off for every C and C++ arm, v2 included. The Rust arms use the harness's release profile, `opt-level = 3`, `lto = true`, `codegen-units = 1`, `panic = "abort"`. It optimizes across all crates, so it is at least cargo's default profile and favours the Rust arms. The Go arms are built as a C archive without profile-guided optimization, which would need a profile of a workload that this design does not have. Appendix Appendix B gives each competitor's configuration and the documentation it follows.

4.3. Arms

An arm is one router behind the harness's common interface. All arms are linked into one binary.

v2. `regexmatcher-v2` adds the routes at run time, builds the table, and looks up through one out-of-line function, `find_v2`. `regexmatcher-v2-ct` declares the same routes as a constant table in a generated translation unit, one per shape and size, aligned to a page. It shares its adapter with `regexmatcher-v2`, takes part in H7 only, and by design refuses tables above 1,000 routes.

Competitors. The sixteen routers of Table 1, each pinned by commit and by the hash of its source archive. Each arm's timed lookup starts from the method and the path, in the form that its library's request carries them. It ends with the route and the captured values, as the router's interface returns them. Steps of a framework outside route lookup, such as pooling a request and writing a response, are left out for every arm. Appendix Appendix A gives each arm's timed call and the differences that remain, because removing them would leave the router's interface.

Reference arms. v1 behind its wrapper (`regexmatcher-v1`), a radix tree of the harness, an exact-match hash table of the harness, and a sequence of `std::regex`. The hash table and the radix tree run in every cell, with as many processes as v2. The other two run up to $m = 10{,}000$, in 3 processes per cell. On the `static` shape, whose one method has only literal routes, v2 answers from its exact-match table, the mechanism of the hash arm.

Null arms. A null arm measures the harness's loop around a lookup that matches nothing: one out-of-line call that reads the path's length. H2 subtracts from each arm the null of its own loop. `null` answers 405 itself, so the harness adds nothing to its answer. It is the null of v2, of the compile-time table and of the ablation arm. A second null arm, `null-no405`, has no 405 of its own, so the harness applies its 405 rule to it, as to every arm without one. It is the null of the C++ competitors, of the Rust competitors behind their C functions and of the reference arms. Each Go arm runs a null pass of its own, in its own process, over the same data.

The ablation arm. `regexmatcher-v2-r1` is the pre-release's header with its namespace and macro prefix renamed and nothing else changed. It runs in every cell of the main grid, decides nothing, and shows what the rework of Section 3.7 changed.

4.4. Tables and Rings

A cell is one arm, one table shape and one table size m. Table 3 lists the shapes. The sizes are $m = 10$, 100, 1,000, 10,000 and 100,000. Literals are short pseudo-words. A route's first literal comes from a small vocabulary and every later one from a large one. A table therefore shares a few top-level words and branches widely below them, as many REST interfaces do. Each process draws its table from its table seed.

Table 3: The seven table shapes. `l` is a literal and `p` a parameter; `*p` is a catch-all.

Shape	**Segments**	**What it tests**
`static`	`/l/l/l`	literal routes only, one method
`param-last`	`/l/l/{p}`	a parameter as the last segment
`param-first`	`/{p}/l/l`	a parameter as the first segment
`rest`	`/l/{p}/l/{q}`	two parameters between literals
`wild`	`/l/l/{*p}`	a catch-all
`mixed-disjoint`	groups of four	a fully literal route and three routes of one resource, `/b/r/{p}`, `/b/r/{p}/s` and `/b/r/{p}/s/{q}`, under different first words
`mixed-overlap`	groups of four	`/t/r/s`, `/t/r/{p}`, `/t/r/s/{p}` and `/t/{p}/u/{q}`: precedence and backtracking

A ring holds 4,096 lookups drawn uniformly over the table's routes, from the process's ring seed. Every lookup in the main grid hits a route. In `mixed-overlap`, half of the queries for the last route of a group carry the literal `r` as the parameter's value. Such a query first descends the literal, fails below it, and must return to the parameter. We call these queries decoys.

4.5. Measurements

Each process builds one table, then measures one arm (`bench/core/runner.hpp`). It reports the median time per lookup over nanobench's epochs. Per lookup it also reports instructions, branch misses, cycles, L1d misses, L2 misses of data requests and L1 DTLB misses, all in user space. The dispatch-stall cycles of every unit-mask bit of AMD's events `0xAE` and `0xAF` are counted, each bit on its own. We found no AMD document that defines these events for L's processor, so they rest on a validation run on L, kept in the paper's repository as `lab/evidence/2026-09-27-L-dispatch-stall-validation`. Latency percentiles come from one lookup at a time. Heap allocations per lookup are counted in a separate pass, never in a timed loop. The process also reports the build time, the bytes allocated by the build and the heap bytes that the built table holds.

The timed loops read a compact copy of the ring, made before the table is built. A Go arm loads its own copy into Go's heap at its first timed pass. Go's collector and system monitor share the measuring core with

the timed loop, and the thread's counters do not count their time. The Go arms' clock, their spread over epochs and their collections during the epochs are therefore in Table S1.

4.6. Checking Every Answer

A time counts only if every answer behind it is right. In each process, an agreement pass checks the route and every captured value of each query against the ring's expectation. The pass stops after 120 s, once it has checked at least 64 queries. After the last measured cell, the run checks every process that the pass cut on every query of the same table and ring. That check is untimed, and its tool is linked from the run's own build. It counts only if its table and ring digests equal the process's. A process that neither check verified on every query fails its cell.

Every process's answers are also judged against declared path semantics. A router whose answers on a shape differ by its own semantics is measured all the same and shown as incomparable, with the range of differing answers. It then takes no part in that shape's cells. Each declaration names the one kind of wrong answer it allows, and any other wrong answer fails the cell. There are four:

- Boost.URL's example router on `mixed-overlap` finds the right route but keeps a value captured in a branch it left.
- gin on `mixed-overlap` answers 404 for each query that needs a return to an earlier parameter. Wherever its route is right, its values are right.
- r3 on `mixed-overlap` answers 404 for such queries, and for more queries as the table grows.
- uWebSockets on `wild` finds the route, but its catch-all returns no value. So uWebSockets takes part in none of the `wild` cells.

An arm that rejects a table while it is built is shown as refused, with the router's reason.

Eight probes test the rules of Section 3.2 on every arm. The most specific route must win in either registration order (two probes). Every pchar must be accepted in a parameter and returned as sent. Bytes must be compared as sent, without decoding. A catch-all must take a non-empty remainder, and an empty segment must not match a parameter. A trailing slash must be strict, and a path that matches only under another method must give 405.

The cut for slow arms. A timed pass that would take longer than 0.25 s is cut to the prefix of the ring that fits, never to fewer than 16 queries. The cut is decided from the agreement pass's time. The ring is in random order, but a cut arm repeats a short prefix, which warms its predictors and caches. A cut arm's time and counters are therefore optimistic for it. The analysis flags every cell where v2 or the fastest competitor was cut. H2 reports the median competitor's branch misses with and without the cut competitors.

Stopping. A process has a budget of 1,800 s. When a cell's first process exceeds it, the cell's other processes do not run, and the cell is reported as over the budget.

4.7. The Sanitizer Gate

A number is citable only if every measured build compiled first-party inputs that green sanitizer records cover. On L the records are AddressSanitizer with UndefinedBehaviorSanitizer, ThreadSanitizer and MemorySanitizer, all with clang 22.1.8. A record covers a build when the hashes of their inputs match, with the same compiler, configuration, pins and fetched archives. RegexMatcher's own suite, with its negative-compilation cases, passed under all three on L. That suite's records cover v2's headers. The harness's records cover every arm, and the server's records cover the server with its end-to-end exercise. Every measured build passed the gate before it ran. The archive of each gated build holds the gate's output, which names the records it matched. Those records are in `lab/sanitizer-records/` of the repository, with records made later from the root commit of its public history that cover the same inputs. The later records used that commit's file of pins. It differs from the runs' file, also in

the repository, only in the pin of v2, which no gated build used, since each gave v2's sources. The repository's gate scripts check both sets of records against each run's archived inputs (`results/round2/records-cover.txt`). The program that hashes a build's inputs is a laboratory tool and is not published; `PROVENANCE.md` states its rule.

MemorySanitizer needs every library in the process instrumented, and the Go and Rust standard libraries are prebuilt and uninstrumented. Its record therefore leaves out the 7 arms behind a foreign-function boundary: matchit, actix-router, path-tree, httprouter, gin, net/http and chi. The gap is declared in the repository. Under AddressSanitizer their Go and Rust code is instrumented too, though not the Rust standard library. Under ThreadSanitizer only their C++ adapters are instrumented. No binary of the records links or calls io_uring, so MemorySanitizer's blind spot for the kernel's writes through it does not apply.

4.8. Seeds and Order

Each cell runs in $R = 16$ processes. Process k uses the kth table seed and the kth ring seed. The table seeds run from 111 to 126, and the ring seeds from 211 to 226. No two pairs share a table or a ring, so the pairs are independent, and the analysis treats them so. It refuses a cell whose v2 processes do not use exactly these pairs. All arms ran in one binary in the same run, and the order of the processes was shuffled with the seed 20261009.

Why $R = 16$: the project's rule on replicates asks for enough of them that an exact paired test can pass the family's Holm threshold. With sixteen independent pairs, the smallest one-sided p of the exact sign test is one half to the power R, that is 1.5×10^{-5}. Holm's first threshold for the family of 35 cells is 0.025 divided by 35, that is 7.1×10^{-4}. One pair at or above the margin gives 2.6×10^{-4}. The smallest p of the bootstrap, about 5.0×10^{-5}, is also below Holm's first threshold. Each table seed is in one pair, so the bootstrap clustered by table seed is the bootstrap over pairs.

4.9. Statistics

A comparison of two arms in a cell is paired by seed pair. Its statistic is the median over the 16 pairs of the ratio of the two arms' times. For H1, the fastest competitor of a cell is the included competitor with the lowest median time. It may differ from cell to cell, and the bootstrap chooses it again in every resample. Two computations, both pre-specified, test each cell.

The BCa interval. 10,000 resamples of the pairs, drawn with replacement, keep each pair's values together. The bias correction counts ties as half, and the acceleration comes from the jackknife over pairs. With sixteen pairs, the leave-one-out medians take two values, eight times each. The acceleration is then zero whenever the fastest competitor is the same in every leave-one-out set. In this run it was zero up to rounding, at most 1.0×10^{-13}, so every interval is the bias-corrected percentile interval. The one-sided p of the null hypothesis is the level α at which the interval's upper bound equals the margin. That interval is two-sided, with coverage one minus twice α.

The exact sign test. X counts the pairs whose ratio is below the margin. A tie counts as not below. Under the null hypothesis that the median ratio is at least the margin, X is binomial with $n = 16$ and probability one half at the boundary, and $p = P(X \geq x)$ exactly.

The implementation uses Python's standard normal distribution. On L it reproduced SciPy's BCa interval to the last digit on the same draws, and SciPy's exact binomial test. One generator per hypothesis, seeded with 20261010, serves the tested cells in the family's order. An untested cell draws nothing.

4.10. The Hypotheses

H1, lookup time. In every cell in which a competitor holds the table, v2 is not slower than the fastest of them. The null hypothesis of a cell is that the ratio is at least 1.02. Holm's step-down runs over the 35 cells at a family-wise 0.025, one-sided, for each computation. A cell passes if it passes under both. A cell that cannot be tested enters Holm with a p of one and does not pass. H1 holds if at least 32 cells pass and the upper bound of the unadjusted 95% BCa interval is at most 1.10 in every tested cell. Superiority, with the margin 1.00, is

tested the same way in a Holm family of its own. It decides nothing about H1. The paper calls v2 faster only in the cells that pass superiority.

H2, work per lookup. In every cell, v2 executes no more instructions per lookup than the fastest competitor, and takes no more branch misses than the median competitor. Instructions are compared net of the null of each arm's own loop, and branch misses raw. Values are medians over the processes, and the comparison is strict. A cell that cannot be judged does not hold.

H3, no allocation. v2 makes no heap allocation per lookup in any process of the 35 cells, captured values included.

H4, build and memory. In every cell, v2's build time is at most the median competitor's, and at $m = 100{,}000$ it is below 1 s. The heap bytes that its table holds are at most 2 times the smallest competitor's. All three must hold in every cell.

H5, checked when compiled, correct when run. (a) Each of the 17 negative-compilation cases fails to compile, with a diagnostic that names its reason. (b) v2 passes every probe. (c) Over HTTP/1.1, in the paper's minimal server, a HEAD request on a path that only GET routes match is served by the GET route without content. A 405's Allow field lists HEAD whenever it lists GET Fielding et al. (2022). HTTP/2 is out of scope, because the server has none.

H6, the client sees the difference. In each of nine cells, the server answers more requests per second with v2 than with v1. That is, the lower bound of the ratio's 95% BCa interval is above 1.00. In each, the measured difference in time per request is at least 0.5 of the difference that the lookup times predict (Section 4.11).

H7, a table built at compile time is not slower. On the same routes and ring, the compile-time table is not slower than v2's table built at run time. In every cell, both computations must pass at the margin 1.02, each at a one-sided 0.025 per cell. The sign test then needs 13 of 16 pairs below the margin. H7 covers the seven shapes at $m \leq 1{,}000$, 21 cells.

What would count against the claim. The claim is that v2 is the fastest route matcher measured here, in the setting of Section 4.2. "Faster" rests on H1's superiority family alone, cell by cell, and only against the competitors included in a cell. H1 or H2 failing in a whole shape narrows the claim to the other shapes, and a loss in a single cell is reported in that cell. If H4, H6 or H7 fails, the paper reports the loss and does not make that hypothesis's claim.

4.11. The Minimal Server and the End-to-End Run

H5(c) and H6 use a minimal HTTP/1.1 server written for this paper, at commit `f626955eb` of its repository. The files that the run compiled are identical to that commit's, as the matching hashes of Section 4.7 show. It runs one worker thread over epoll with keep-alive, and answers pipelined requests in order. Each route answers with a fixed body of 13 bytes. HEAD and the Allow field live above the router, so every router arm runs the same code for them. One binary holds three router arms behind one function pointer: v1 behind its wrapper, v2, and a null router that answers without reading the path. The server is not a full framework. It parses requests, routes them and writes fixed responses, and nothing else. Section 5.8 records the absent end-to-end comparison with the external routers.

H6 has nine cells: the seven shapes at $m = 10$, and `rest` and `param-last` at $m = 10{,}000$. Each serves the routes of table seed 111. Its targets are one per route, with each parameter filled as ring seed 211 fills it, in shuffled order. The server runs on CPU 14, whose sibling CPU 15 idles. The load generator runs on CPUs 2 to 13, over 64 connections over loopback, with one request in flight on each. Each window starts a fresh server, probes it, warms it up for 1 s and measures for 5 s. Requests per second are the completed successful responses over the measured wall time. The load generator and the program that runs the windows are the laboratory's T1 tools. Their sources, as the run used them, are in `lab/t1/` of the repository. The load generator's own sanitizer record is a laboratory record and is not published.

The arms run in mirrored rounds of v1, v2, null, null, v2 and v1, and one round is one pair. A window is invalid if its server did not start or answer its probe, or if it completed no request. It is also invalid if its error

share exceeded 0.1%, if the generator's CPUs were more than 90% busy, or if the clock drifted more than 2%. A round with an invalid window is not a valid pair. A cell runs at least 6 and at most 20 rounds. It stops once a bootstrap interval of the geometric mean ratio, for v2 and for the null arm against v1, lies outside $[0.98, 1.02]$. It also stops once that interval's half-width is below 1%.

Per pair, each arm's rate X is the mean over its two windows, and its time per request τ is the reciprocal of X. The ratio is X_{v2}/X_{v1}, and the measured difference is $D_{\text{meas}} = \tau_{v1} - \tau_{v2}$. The lookup difference $D_{T0} = t_{v1} - t_{v2}$ is v1's time minus v2's in the main grid's process at pair (111, 211), in the same cell. H6(a) needs the lower bound of the ratio's 95% BCa interval above 1.00 in all nine cells. H6(b) needs the lower bound of the interval of D_{meas}/D_{T0} at least 0.5 in all nine. D_{T0} is one value per cell, so the interval of (b) covers the variation of the end-to-end run only. The bound 0.5 was chosen before the run, informed by development data from an earlier build that the paper does not report. With at least 6 pairs, a cell's exact sign test can reach 1.6×10^{-2}. Beside each cell, and untested, we report the null arm's time per request. We also report the ratio that the lookup times predict, $P = (\tau_{\text{null}} + t_{v1})/(\tau_{\text{null}} + t_{v2})$.

4.12. Exploratory Runs

Each exploratory run used the main grid's gated build and order seed, on the first R pairs. None has an interval, and none decides anything.

- Miss-heavy rings, where half the queries match no route: every cell, $R = 3$.
- The GitHub interface table of the Go benchmark suite Schmidt (2020), with 203 routes and four methods: one cell, $R = 16$.
- Long literals of 12 to 24 letters at $m = 10{,}000$ and 100,000, $R = 3$. v2 keys a segment on its first 8 bytes, and short literals might favour it.
- A Zipf-distributed ring in the same cells, $R = 3$.
- `mixed-overlap` without decoys at every size, $R = 16$, where gin takes part because its walk then never needs to return.
- The ablation arm in every cell of the main grid.

The pre-specification named one more run, for an H7 cell with a table seed apart from the others. It measures that cell again, with the compile-time table copied into a heap block in the same binary. Its condition held, and the run was not made in the frozen order. It was made afterwards, as an exploratory run, gated by sanitizer records of its own (Section 5.9).

4.13. Deviations and the Revision Log

The revision log holds five entries. The first is a deviation, and it concerns the check on every query at the end of the main grid. The frozen text fixed the check, but not how many checks run at once. The run started 16 at once. The actix-router checks at $m = 100{,}000$ took up to 2 GB each and exhausted L's 15 GB. The kernel killed 79 of the 115 checks, all after the last measured cell, and none had reported a wrong answer.

The repeat of all 115 checks, with 4 at once, was logged before it ran, because it could change H1's decision. It used the same checking binary from the run's own build, after its hash was checked, and wrote a separate file. The first file was kept. The repeat checked answers only: nothing was timed again, and no time was replaced. The later exploratory grids ran their checks 4 at once. The repeat changed which cells could be tested, as Section 5.2 shows.

The other four entries change no decision. The second records that the paper's numbers come from two generated files. One holds the analysis's macros; the other was written after the run, from the analysis's outputs

and the raw data. The third records that the conditional run of Section 4.12 was due, was not made in the frozen order, and was made afterwards. The fourth records how the paper mentions the development round: the origin of H6's bound, and the direction of one cell, without its numbers. The fifth records that the conditional run used a fresh build, which adds its one arm and changes no file that the measured arms compile. One more departure is not in the revision log. A few seconds of analysis ran on L at the lowest priority while the main grid measured. No process was excluded for it.

5. Results

5.1. The Run

The main grid measured 13,048 processes of 25 arms. Of these, 11,864 ended with every answer right, 864 were refusals of a table, and 320 differed from the expected answers as their arm's declaration allows. No cell failed and no process exceeded its budget. The agreement pass stopped 115 processes at its budget, 96 of them actix-router at $m = 100{,}000$. The repeated check of Section 4.13 found all 115 right on every query.

A competitor was absent from a cell only by refusal or by declaration. On `wild`, uWebSockets is incomparable and Pistache refuses the tables, because its splat matches one segment only. So 14 competitors take part. On `mixed-overlap`, Boost.URL, gin and r3 are incomparable and httprouter refuses the tables, so 12 take part. Elsewhere all sixteen do. In no cell of H1 did v2 or the fastest competitor run on a prefix of the ring.

5.2. H1: Lookup Time

H1 holds. Of the 35 cells, 34 pass under both computations, where 32 were needed, and no tested cell's upper bound exceeds 1.10. The largest is 1.0830. The two computations agree in every cell. Of the cells that pass, 33 also pass the superiority test. In those cells, and in the setting of Section 4.2, v2 is faster than the fastest competitor. There it took 0.1691 to 0.9053 of that competitor's time. Figure 1 shows every cell, and Table 4 gives the numbers.

gin was the fastest competitor in 25 cells, matchit in 6, uWebSockets in 3 and glaze in 1. The gap grows with the table. On `static`, v2's time rose from 19.7 ns at $m = 10$ to 31.2 ns at $m = 100{,}000$. Over the same sizes, the fastest competitor's rose from 34.6 ns to 183.2 ns.

***`wild`** at $m = 10$.* This is the one cell that does not pass. Its ratio against gin is 1.0262, with the interval [0.9906, 1.0830]. Only 8 of 16 pairs are below 1.02, and the Holm-adjusted p is 6.0×10^{-1}. So the paper claims neither "not slower" nor "faster" in this cell. The loss is in a single cell, so the claim narrows at that cell only, as the pre-specification requires. `wild` passes both tests at every other size. Section 6.2 discusses this cell.

***`mixed-disjoint`** at $m = 10$.* This cell passes non-inferiority and not superiority. Its ratio against gin is 0.9730, with the interval [0.9530, 1.0156]. Of the sixteen pairs, 13 are below 1.02, but only 10 below 1.00. So v2 is not slower than gin here, and the paper does not call it faster.

Whether this cell passes depends on the family that Holm's procedure adjusts it in. Before the repeated check, 6 cells at $m = 100{,}000$ could not be tested, because processes of actix-router were not verified on every query. Each entered Holm with a p of one, and 27 of the 29 tested cells passed. This cell's raw sign-test p is 1.1×10^{-2} in both analyses. With the untested cells at the bottom of the order, its Holm multiplier was 8, so its adjusted p was 8.5×10^{-2}, above 0.025. Once those cells were tested, their p were small. The cell's multiplier fell to 2, and its adjusted p to 2.1×10^{-2}. Its bootstrap p moved as well, from 4.2×10^{-3} to 2.4×10^{-3} before adjustment, and from 3.3×10^{-2} to 4.7×10^{-3} after it. One generator serves the tested cells in the family's order, so more tested cells change the draws of every later cell. The cell's ratio and interval do not depend on the family.

5.3. H2: Work per Lookup

H2 does not hold. It holds in 33 of 35 cells (Table 5). v2 took fewer branch misses than the median competitor in every cell, with or without the cut competitors, `actix-router`, `cpp-httplib`, `drogon` and `oatpp`.

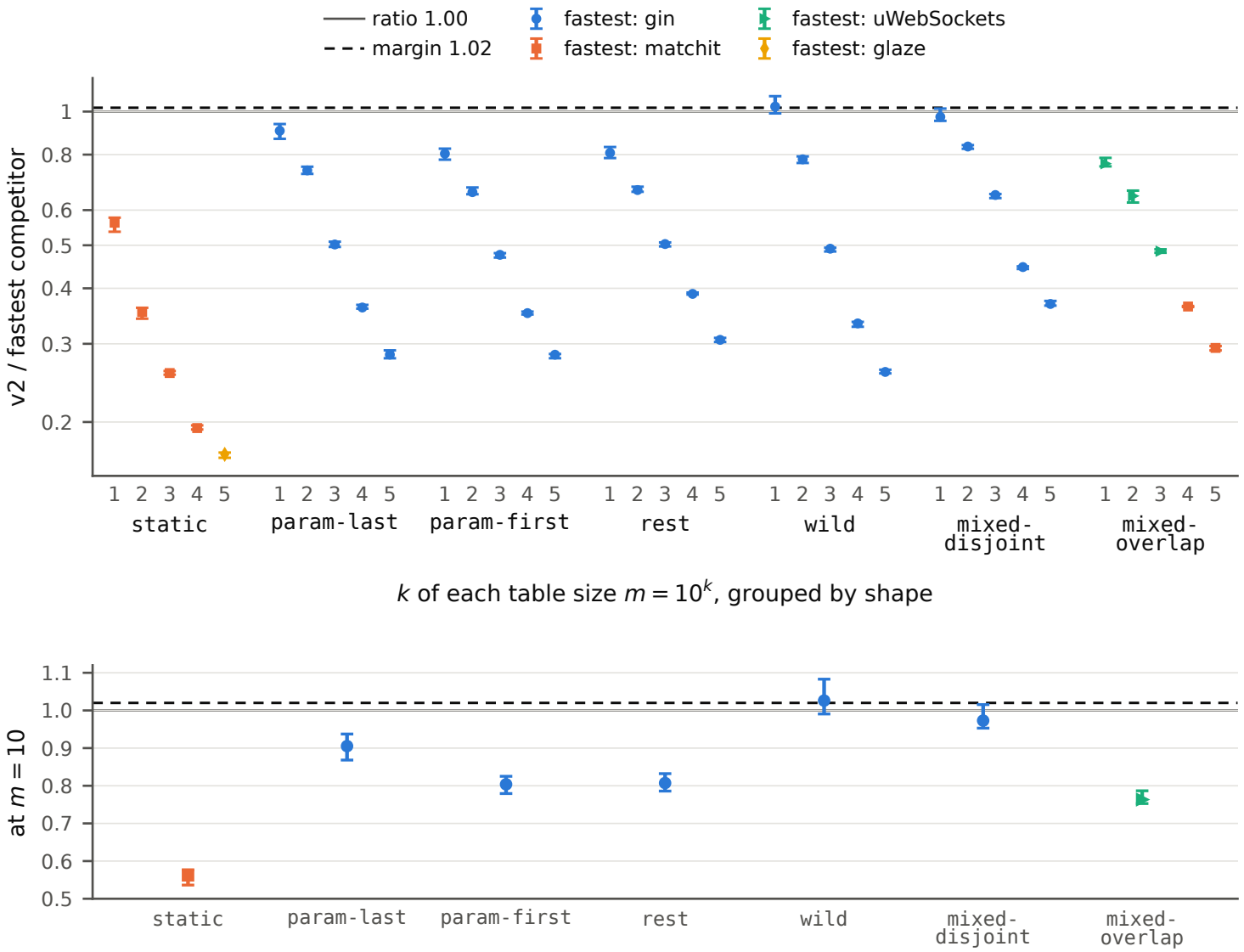


Figure 1: H1 per cell: v2's time over the fastest competitor's, as the median over the 16 pairs of the per-pair ratio, with its 95% BCa interval. Colour and marker name the fastest competitor. The dashed line is the margin 1.02, and the solid line the ratio 1.00. The upper panel shows every cell on a logarithmic axis. The lower panel shows the cells at $m = 10$ on a linear axis, where the two lines are told apart.

Table 4: H1 per cell. n is the number of competitors in the cell. v2 and F are the medians over processes of v2's and the fastest competitor's time per lookup. "Ratio" is the median of the per-pair ratios, with its 95% BCa interval. X counts the pairs below 1.02. p_{NI} and p_{S} are the larger of the two Holm-adjusted p of non-inferiority and of superiority. NI and S say whether the cell passes each.

Shape	m	**Fastest**	n	**v2 (ns)**	**F (ns)**	**Ratio**	**95% BCa**	X	p_{NI}	**NI**	p_{S}	**S**
`static`	10	`matchit`	16	19.7	34.6	0.5621	[0.5363, 0.5769]	16	1.2×10^{-3}	yes	1.2×10^{-3}	yes
	100	`matchit`	16	20.2	56.8	0.3537	[0.3417, 0.3616]	16	1.2×10^{-3}	yes	1.2×10^{-3}	yes
	1,000	`matchit`	16	21.4	82.9	0.2578	[0.2556, 0.2605]	16	5.3×10^{-4}	yes	5.3×10^{-4}	yes
	10,000	`matchit`	16	27.5	140.7	0.1938	[0.1924, 0.1965]	16	1.2×10^{-3}	yes	1.2×10^{-3}	yes
	100,000	`glaze`	16	31.2	183.2	0.1691	[0.1661, 0.1707]	16	1.2×10^{-3}	yes	1.2×10^{-3}	yes
`param-last`	10	`gin`	16	48.0	51.1	0.9053	[0.8682, 0.9374]	16	1.2×10^{-3}	yes	1.2×10^{-3}	yes
	100	`gin`	16	55.3	75.1	0.7371	[0.7240, 0.7513]	16	1.2×10^{-3}	yes	1.2×10^{-3}	yes
	1,000	`gin`	16	52.1	103.3	0.5021	[0.4958, 0.5093]	16	1.2×10^{-3}	yes	1.2×10^{-3}	yes
	10,000	`gin`	16	68.3	187.6	0.3621	[0.3600, 0.3671]	16	1.2×10^{-3}	yes	1.2×10^{-3}	yes
	100,000	`gin`	16	72.3	256.2	0.2836	[0.2783, 0.2900]	16	1.2×10^{-3}	yes	1.2×10^{-3}	yes
`param-first`	10	`gin`	16	46.8	56.5	0.8038	[0.7793, 0.8251]	16	1.2×10^{-3}	yes	1.2×10^{-3}	yes
	100	`gin`	16	53.3	81.0	0.6586	[0.6514, 0.6747]	16	1.2×10^{-3}	yes	1.2×10^{-3}	yes
	1,000	`gin`	16	51.0	108.0	0.4758	[0.4692, 0.4800]	16	1.2×10^{-3}	yes	1.2×10^{-3}	yes
	10,000	`gin`	16	66.0	187.8	0.3517	[0.3493, 0.3545]	16	1.2×10^{-3}	yes	1.2×10^{-3}	yes
	100,000	`gin`	16	71.9	254.5	0.2835	[0.2784, 0.2846]	16	1.2×10^{-3}	yes	1.2×10^{-3}	yes
`rest`	10	`gin`	16	59.6	72.1	0.8072	[0.7859, 0.8322]	16	1.2×10^{-3}	yes	1.2×10^{-3}	yes
	100	`gin`	16	66.6	99.9	0.6658	[0.6598, 0.6773]	16	1.2×10^{-3}	yes	1.2×10^{-3}	yes
	1,000	`gin`	16	64.5	128.3	0.5032	[0.4971, 0.5074]	16	1.2×10^{-3}	yes	1.2×10^{-3}	yes
	10,000	`gin`	16	81.4	209.5	0.3886	[0.3873, 0.3913]	16	1.2×10^{-3}	yes	1.2×10^{-3}	yes
	100,000	`gin`	16	84.2	274.9	0.3062	[0.3029, 0.3093]	16	1.2×10^{-3}	yes	1.2×10^{-3}	yes
`wild`	10	`gin`	14	49.5	46.2	1.0262	[0.9906, 1.0830]	8	6.0×10^{-1}	no	9.6×10^{-1}	no
	100	`gin`	14	56.8	73.1	0.7806	[0.7661, 0.7926]	16	1.2×10^{-3}	yes	1.2×10^{-3}	yes
	1,000	`gin`	14	53.9	110.4	0.4912	[0.4842, 0.4937]	16	1.2×10^{-3}	yes	1.2×10^{-3}	yes
	10,000	`gin`	14	68.4	205.0	0.3335	[0.3277, 0.3361]	16	1.2×10^{-3}	yes	1.2×10^{-3}	yes
	100,000	`gin`	14	70.3	270.5	0.2593	[0.2574, 0.2623]	16	1.2×10^{-3}	yes	1.2×10^{-3}	yes
`mixed-disjoint`	10	`gin`	16	51.8	53.4	0.9730	[0.9530, 1.0156]	13	2.1×10^{-2}	yes	4.5×10^{-1}	no
	100	`gin`	16	62.6	75.1	0.8342	[0.8246, 0.8403]	16	1.2×10^{-3}	yes	1.2×10^{-3}	yes
	1,000	`gin`	16	66.6	103.1	0.6488	[0.6386, 0.6522]	16	1.2×10^{-3}	yes	1.2×10^{-3}	yes
	10,000	`gin`	16	83.4	186.7	0.4466	[0.4422, 0.4482]	16	1.2×10^{-3}	yes	1.2×10^{-3}	yes
	100,000	`gin`	16	92.7	251.1	0.3688	[0.3657, 0.3747]	16	1.2×10^{-3}	yes	1.2×10^{-3}	yes
`mixed-overlap`	10	`uwebsockets`	12	52.8	68.8	0.7633	[0.7527, 0.7866]	16	1.2×10^{-3}	yes	1.2×10^{-3}	yes
	100	`uwebsockets`	12	63.8	100.5	0.6456	[0.6239, 0.6638]	16	1.2×10^{-3}	yes	1.2×10^{-3}	yes
	1,000	`uwebsockets`	12	76.9	158.6	0.4848	[0.4810, 0.4896]	16	1.2×10^{-3}	yes	1.2×10^{-3}	yes
	10,000	`matchit`	12	90.7	250.0	0.3640	[0.3630, 0.3650]	16	1.2×10^{-3}	yes	1.2×10^{-3}	yes
	100,000	`matchit`	12	103.8	353.8	0.2937	[0.2899, 0.2965]	16	1.2×10^{-3}	yes	1.2×10^{-3}	yes

The 2 losses are net instructions against gin, at `param-last` at $m = 10$ and `mixed-disjoint` at $m = 10$. At `param-last`, v2 ran 470.2 net instructions per lookup against gin's 464.3. At `mixed-disjoint`, it ran 517.8 against 479.3. The pre-specification predicted the second loss and not the first.

The memory-side counters per lookup of the losing cells are small for both arms. At `param-last`, v2 took 1.01 L1d misses, 0.0021 L2 misses and 0.02 DTLB misses. gin took 1.01, 0.0053 and 0.01. At `mixed-disjoint`, v2 took 1.09, 0.0071 and 0.02, and gin 1.15, 0.0073 and 0.01. In both cells v2 still took fewer branch misses than the median competitor, and by H1 it was not slower than gin.

Table 5: H2 per cell: instructions per lookup, net of the null of each arm's own loop and raw, and branch misses per lookup. "Median" is the median competitor's branch misses. The next column leaves out the competitors whose passes were cut, where any were.

			Net instructions		**Raw instructions**		**Branch misses**			
Shape	m	**Fastest**	**v2**	**Fastest**	**v2**	**Fastest**	**v2**	**Median**	**Uncut**	**Holds**
`static`	10	`matchit`	214.2	252.8	243.2	281.8	0.42	1.27		yes
	100	`matchit`	214.5	366.5	243.5	395.5	0.45	2.83		yes
	1,000	`matchit`	216.3	439.8	245.3	468.8	0.50	4.99		yes
	10,000	`matchit`	214.0	518.5	243.0	547.6	0.41	6.92	6.29	yes
	100,000	`glaze`	214.8	736.2	243.8	765.2	0.43	8.58	7.53	yes
`param-last`	10	`gin`	470.2	464.3	499.2	510.5	1.54	2.37		no
	100	`gin`	476.8	620.0	505.8	666.1	2.21	5.26		yes
	1,000	`gin`	486.5	727.4	515.5	773.6	1.77	6.47	6.12	yes
	10,000	`gin`	486.1	841.3	515.1	887.4	1.75	8.64	7.03	yes
	100,000	`gin`	486.6	952.9	515.6	999.1	1.80	10.04	8.44	yes
`param-first`	10	`gin`	468.1	529.4	497.1	575.6	1.36	2.91		yes
	100	`gin`	474.9	685.4	503.9	731.5	2.08	5.40		yes
	1,000	`gin`	484.4	793.1	513.4	839.2	1.66	6.67	6.51	yes
	10,000	`gin`	483.5	905.1	512.5	951.2	1.65	8.88	7.07	yes
	100,000	`gin`	484.2	1,008.1	513.2	1,054.2	1.71	9.94	8.51	yes
`rest`	10	`gin`	579.8	672.7	608.8	718.9	2.00	5.14		yes
	100	`gin`	586.0	828.7	615.0	874.8	2.71	9.83		yes
	1,000	`gin`	594.9	937.5	623.9	983.7	2.26	11.20	10.39	yes
	10,000	`gin`	595.1	1,051.0	624.1	1,097.1	2.27	12.24	11.05	yes
	100,000	`gin`	596.1	1,162.3	625.1	1,208.5	2.36	13.18	11.84	yes
`wild`	10	`gin`	457.0	462.9	486.0	509.0	1.70	3.16		yes
	100	`gin`	462.8	619.8	491.8	665.9	2.41	8.82		yes
	1,000	`gin`	472.5	726.7	501.5	772.8	2.00	15.87	6.78	yes
	10,000	`gin`	472.1	839.5	501.1	885.7	1.97	15.93	8.02	yes
	100,000	`gin`	473.2	948.3	502.2	994.4	2.06	15.63	10.10	yes
`mixed-disjoint`	10	`gin`	517.8	479.3	546.8	525.4	1.61	2.71		no
	100	`gin`	539.4	626.8	568.4	673.0	2.65	6.03		yes
	1,000	`gin`	550.4	729.1	579.4	775.2	2.84	10.84	10.64	yes
	10,000	`gin`	551.6	855.8	580.7	901.9	2.88	10.88	10.18	yes
	100,000	`gin`	552.9	958.6	581.9	1,004.8	2.98	11.28	10.83	yes
`mixed-overlap`	10	`uwebsockets`	531.9	642.0	560.9	671.0	1.58	2.98		yes
	100	`uwebsockets`	573.0	808.3	602.0	837.4	2.49	8.10		yes
	1,000	`uwebsockets`	591.7	1,040.4	620.7	1,069.4	3.44	11.78	11.59	yes
	10,000	`matchit`	589.9	1,065.2	618.9	1,094.2	2.87	13.74	10.92	yes
	100,000	`matchit`	591.3	1,140.5	620.3	1,169.5	2.97	25.00	11.43	yes

5.4. H3: No Allocation

H3 holds. In all 560 processes of v2 over the 35 cells, a lookup made no heap allocation. The compile-time arm, in its 336 processes, and the ablation arm, in its 560, made none either.

5.5. H4: Build Time and Memory

H4 does not hold. It fails in 4 cells, all on build time: `static` and `param-first` at $m = 10{,}000$ and 100,000. There v2's build took 1.07 to 1.28 times the median competitor's (Table 6). The other two bounds hold in every cell. v2's slowest build at $m = 100{,}000$ took 156 ms, below 1 s. Its table held 0.65 to 1.26 times the heap bytes of the smallest competitor, within the factor 2. As pre-specified, H4's claim is not made.

Table 6: H4 per cell. Build times are medians over processes of each process's median build, in microseconds. "Median" is the median competitor's build time. Heap bytes are those the built table holds. "Smallest" is the competitor with the fewest.

		Build (μs)		Heap bytes			Holds		
Shape	m	**v2**	**Median**	**v2**	**Smallest**		**Build**	**Heap**	**Cell**
`static`	10	4.00	6.92	2,104	2,264	`httprouter`	yes	yes	yes
	100	41.9	60.6	13,896	16,032	`crow`	yes	yes	yes
	1,000	578	657	112,344	145,080	`crow`	yes	yes	yes
	10,000	9,030	7,080	1,663,304	1,545,724	`crow`	no	yes	no
	100,000	138,000	125,000	13,709,640	14,533,324	`crow`	no	yes	no
`param-last`	10	5.50	10.8	3,136	3,040	`crow`	yes	yes	yes
	100	51.5	78.7	17,800	23,628	`crow`	yes	yes	yes
	1,000	664	866	182,456	223,128	`crow`	yes	yes	yes
	10,000	10,300	11,600	2,151,032	2,352,384	`crow`	yes	yes	yes
	100,000	153,000	154,000	18,598,456	22,559,624	`crow`	yes	yes	yes
`param-first`	10	5.17	9.40	2,880	2,312	`crow`	yes	yes	yes
	100	49.5	69.8	14,632	14,968	`crow`	yes	yes	yes
	1,000	606	736	150,456	135,176	`crow`	yes	yes	yes
	10,000	9,140	8,550	1,831,032	1,453,140	`crow`	no	yes	no
	100,000	151,000	133,000	15,401,288	12,464,636	`crow`	no	yes	no
`rest`	10	6.27	12.4	3,456	4,320	`crow`	yes	yes	yes
	100	57.0	101	18,824	29,016	`crow`	yes	yes	yes
	1,000	859	1,010	183,480	228,776	`crow`	yes	yes	yes
	10,000	10,700	13,000	2,152,056	2,358,040	`crow`	yes	yes	yes
	100,000	150,000	168,000	18,599,480	22,565,272	`crow`	yes	yes	yes
`wild`	10	5.46	11.3	3,136	3,040	`crow`	yes	yes	yes
	100	52.0	94.1	17,800	23,628	`crow`	yes	yes	yes
	1,000	636	1,010	182,456	223,176	`crow`	yes	yes	yes
	10,000	10,300	11,700	2,151,032	2,352,472	`crow`	yes	yes	yes
	100,000	156,000	199,000	18,598,456	22,559,648	`crow`	yes	yes	yes
`mixed-disjoint`	10	5.68	9.88	2,368	2,488	`crow`	yes	yes	yes
	100	59.7	86.2	15,976	17,216	`crow`	yes	yes	yes
	1,000	671	834	144,392	161,692	`crow`	yes	yes	yes
	10,000	8,980	9,380	1,713,072	1,651,436	`crow`	yes	yes	yes
	100,000	120,000	140,000	14,889,744	16,027,164	`crow`	yes	yes	yes
`mixed-overlap`	10	5.40	8.29	2,136	2,672	`crow`	yes	yes	yes
	100	54.0	87.1	15,456	20,536	`crow`	yes	yes	yes
	1,000	633	843	128,560	179,236	`crow`	yes	yes	yes
	10,000	8,500	9,070	1,630,088	1,836,628	`crow`	yes	yes	yes
	100,000	105,000	138,000	14,288,480	17,764,948	`crow`	yes	yes	yes

5.6. H5: Checked When Compiled, Correct When Run

H5 holds in all three parts. (a) RegexMatcher's suite at the measured commit holds the 17 negative-compilation cases, one test each. Each expects the compilation to fail and checks that the diagnostic names the reason. Under each of the three sanitizers on L, all 87 tests of the suite passed. (b) v2 answered all 39 lookups of the probes as the rules require. Table 7 shows every arm. v1 behind its wrapper fails two probes; this decides nothing. Of the competitors, only matchit and path-tree pass every probe, as v2 does. (c) The minimal server's suite checks that the GET route serves HEAD on a path that only GET routes match, without content. It also checks that a HEAD route wins over that fallback, and that Allow lists HEAD whenever it lists GET. The suite passed under each of the three sanitizers on L.

Table 7: The probes of the path rules on every arm. S→P and P→S register a literal route before a parameter route and after it. "pchar" sends every pchar in a parameter, "raw" a path that differs only in the case of a percent escape. "catch-all" and "empty" test the rules on remainders and empty segments, "slash" a trailing slash, and 405 a path of another method. ✓: every lookup right; ×: at least one wrong; ○: the arm refused the probe's routes.

Arm	S→P	P→S	pchar	raw	catch-all	empty	slash	405
`regexmatcher-v2`	✓	✓	✓	✓	✓	✓	✓	✓
`regexmatcher-v1`	✓	×	×	✓	✓	✓	✓	✓
`actix-router`	✓	×	✓	✓	×	✓	✓	✓
`boost-url`	✓	✓	✓	×	×	×	✓	✓
`chi`	✓	✓	✓	✓	×	✓	✓	✓
`cpp-httplib`	✓	×	✓	✓	✓	×	×	✓
`crow`	✓	×	✓	✓	✓	✓	✓	✓
`drogon`	✓	✓	✓	×	×	×	✓	×
`gin`	✓	✓	✓	✓	×	✓	✓	✓
`glaze`	✓	✓	×	✓	×	✓	×	✓
`httprouter`	○	○	✓	✓	×	✓	✓	✓
`matchit`	✓	✓	✓	✓	✓	✓	✓	✓
`nethttp`	✓	✓	×	×	×	✓	✓	✓
`oatpp`	✓	×	✓	✓	×	✓	×	✓
`path-tree`	✓	✓	✓	✓	✓	✓	✓	✓
`pistache`	✓	✓	✓	✓	○	✓	×	✓
`r3`	✓	×	✓	✓	×	✓	✓	×
`uwebsockets`	✓	✓	✓	✓	×	✓	✓	✓

5.7. H6: The Client Sees the Difference

H6 holds. Part (a) holds in 9 of 9 cells, and part (b) holds in 9 of 9 (Table 8). Every cell stopped at 6 valid pairs, and none of the 324 windows was invalid. With v2 the server answered 120,168 to 121,704 requests per second in every cell. At $m = 10$ the lower bounds of the ratio over v1 ranged from 1.0465 to 1.4208. At `rest` with $m = 10{,}000$, v1's lookup dominated its server's time, and the ratio was 68.3570. There v1's lookup time in the main grid rests on a prefix of 499 of the 4,096 queries, as the pre-specification allows.

The lower bounds of the carry-over ranged from 0.9228 to 1.9348, all above 0.5. The null arm took 8.16 to 8.21 μs per request. The measured ratio was 0.9962 to 1.1110 times the ratio that the lookup times predict. In every cell every pair favoured v2 and carried at least 0.5, so each sign test gives 1.6×10^{-2}. H6 compares v2 with v1 in the paper's own server. It says nothing about the competitors, nor about a full framework. Section 5.8 records the comparison that is still missing.

5.8. End to End Against the External Routers

The measurement plan asks for end-to-end latency against the external routers, at the same table sizes as the lookup grid. The archives of this paper record no such run for gin, matchit or any other external router. H6, above, compares v2 with v1 inside this paper's server, and that comparison says nothing about the competitors. This manuscript leaves the block out. No request rate against those routers is reported here.

Table 8: H6 per cell, over loopback with the minimal server. Rates are the medians over pairs, in requests per second. "Low" is the lower bound of the 95% BCa interval. D_{T0} and D_{meas} are in ns per request. The last column is the measured ratio over the predicted one.

Shape	m	**Pairs**	**v1**	**v2**	**v2/v1**	**Low**	D_{T0}	D_{meas}	D_{meas}/D_{T0}	**Low**	**Meas./pred.**
`static`	10	6	115,813	121,704	1.0500	1.0465	197.3	411.5	2.0860	1.9348	1.0253
`param-last`	10	6	109,073	121,038	1.1102	1.1010	662.3	909.2	1.3726	1.2615	1.0275
`param-first`	10	6	84,502	121,026	1.4305	1.4208	2,643.9	3,562.8	1.3475	1.3172	1.0827
`rest`	10	6	104,414	120,180	1.1525	1.1465	1,236.2	1,269.7	1.0271	0.9888	1.0026
`wild`	10	6	104,388	120,476	1.1556	1.1488	1,172.8	1,290.7	1.1006	1.0548	1.0118
`mixed-disjoint`	10	6	110,732	120,866	1.0923	1.0886	795.3	764.4	0.9612	0.9228	0.9962
`mixed-overlap`	10	6	103,717	120,168	1.1598	1.1547	1,134.8	1,327.7	1.1700	1.1349	1.0195
`rest`	10,000	6	1,759	120,361	68.3570	67.8940	501,427.7	560,003.1	1.1168	1.1107	1.1110
`param-last`	10,000	6	97,456	120,586	1.2364	1.2334	1,074.6	1,962.3	1.8260	1.7981	1.0937

5.9. H7: A Table Built at Compile Time

H7 does not hold. The compile-time table passed its per-cell test in 20 of 21 cells, and the rule needs all of them (Figure 2 and Table 9). The bootstrap passed in all 21 cells, and the largest upper bound was 1.0150. The sign test failed in one cell, `param-last` at $m = 1{,}000$. There only 12 pairs are below 1.02, where 13 are needed, and p is 3.8×10^{-2}. As pre-specified, H7's claim is not made.

In that cell the compile-time table is measurably slower, by a small margin. Its median ratio is 1.0048, and its 95% BCa interval, [1.0008, 1.0129], lies wholly above one. Of the sixteen pairs, 12 are above one and 4 reach the margin, where the rule allows 3. One of them, at table seed 124, has the ratio 1.1595. There the compile-time table took 191.4 cycles per lookup against 165.2, with equal instructions. The largest ratio of the other pairs is 1.0351. This pair is an observation only, and it does not explain the cell.

The heap-block follow-up. For such a cell, the pre-specification named an exploratory run with the compile-time table copied into a heap block, to test whether the table's place in memory is the cause. It was made after the main grid, not in the frozen order, and it decides nothing. One binary held the run-time table, the compile-time table and the heap copy, on the same 16 pairs. There table seed 124 no longer stood apart. Its compile-time table took 1.0126 of the run-time table's time, and its heap copy 0.9950. Over the other pairs, the heap copy took 0.9117 to 1.0685 of the run-time table's time, with the median 0.9939.

Single slow pairs appeared again, at other seeds and in each arm. The compile-time table took 1.1512 of the run-time table's time at table seed 114. The heap copy took 1.0685 at table seed 125. The run-time table ran 183.3 cycles per lookup at table seed 122, against its median of 165.6. The heap copy took as many L1d misses per lookup as the run-time table, 4.07 against 4.06, and the compile-time table fewer, 1.06, while their times stayed alike. So a slow pair followed neither the table seed nor the table's place in memory. The run does not identify what slowed those pairs.

The costs of a compile-time table. Table 10 reports what the tables cost the compilers. On L, clang 22.1.8 compiled each of the 358 tables in a fresh build, one job at a time, at the native flags. These are the 119 tables of each size, from the sixteen table seeds of the run and one more, and the GitHub table. A table's object size is the sum of its sections named `.text*`, `.rodata*`, `.data.rel.ro*`, the other `.data*`, and `.bss*`. Each object file is 5,715 to 5,731 bytes larger. The page alignment of the table pads the file, without adding memory that the lookup touches.

Clang's need of constant-evaluation steps was measured for the tables of table seed 111 and for the GitHub table. The search doubles the budget from 1,048,576 steps, so each value is the smallest budget tried under which a table compiled. Every table of 1,000 routes compiled within 16,777,216 steps and failed at 8,388,608. The budget of 33,554,432 is twice the largest of these. On W, an AMD Ryzen 5 3600 (Zen 2), MSVC 19.51.36246 compiled the same 22 tables one at a time, with the flags of RegexMatcher's own test of compile-time tables. Its search starts at 1,048,576 steps and multiplies by four. At 1,000 routes every table crashed the compiler

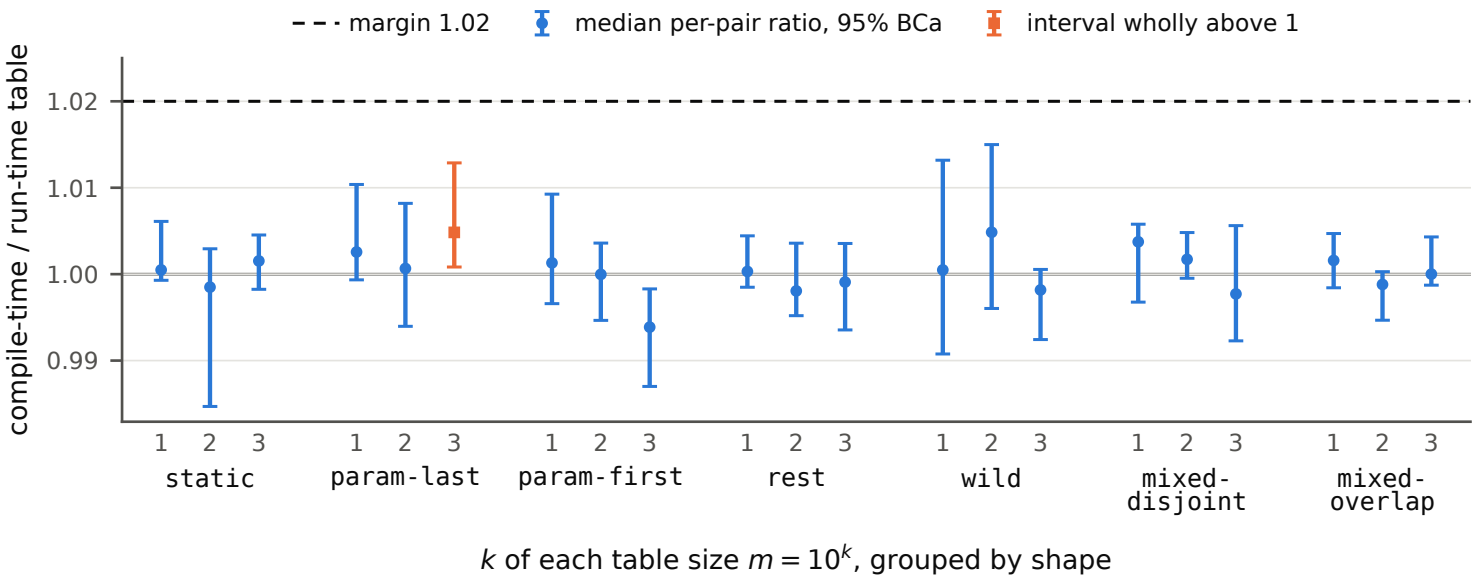


Figure 2: H7 per cell: the compile-time table's time over the run-time table's, as the median of the per-pair ratios, with its 95% BCa interval. A square marks a cell whose whole interval lies above one. The dashed line is the margin 1.02.

Table 9: H7 per cell. CT and RT are the medians over processes of the compile-time and run-time tables' times per lookup. "Upper" is the upper bound of the 95% BCa interval. X counts the pairs below 1.02, and p is the exact sign test's.

Shape	m	CT (ns)	RT (ns)	Ratio	Upper	X	p	Boot.	Sign	Holds
`static`	10	19.8	19.7	1.0005	1.0061	15	2.6×10^{-4}	yes	yes	yes
	100	20.0	20.2	0.9985	1.0029	14	2.1×10^{-3}	yes	yes	yes
	1,000	21.5	21.4	1.0015	1.0045	16	1.5×10^{-5}	yes	yes	yes
`param-last`	10	49.2	48.0	1.0026	1.0104	14	2.1×10^{-3}	yes	yes	yes
	100	55.5	55.3	1.0007	1.0082	16	1.5×10^{-5}	yes	yes	yes
	1,000	52.4	52.1	1.0048	1.0129	12	3.8×10^{-2}	yes	no	no
`param-first`	10	46.3	46.8	1.0013	1.0093	13	1.1×10^{-2}	yes	yes	yes
	100	53.4	53.3	1.0000	1.0036	16	1.5×10^{-5}	yes	yes	yes
	1,000	50.7	51.0	0.9939	0.9983	16	1.5×10^{-5}	yes	yes	yes
`rest`	10	59.4	59.6	1.0003	1.0044	14	2.1×10^{-3}	yes	yes	yes
	100	66.7	66.6	0.9981	1.0036	15	2.6×10^{-4}	yes	yes	yes
	1,000	64.4	64.5	0.9991	1.0036	16	1.5×10^{-5}	yes	yes	yes
`wild`	10	51.2	49.5	1.0005	1.0132	14	2.1×10^{-3}	yes	yes	yes
	100	57.4	56.8	1.0048	1.0150	13	1.1×10^{-2}	yes	yes	yes
	1,000	53.8	53.9	0.9982	1.0005	15	2.6×10^{-4}	yes	yes	yes
`mixed-disjoint`	10	51.8	51.8	1.0037	1.0058	15	2.6×10^{-4}	yes	yes	yes
	100	62.8	62.6	1.0017	1.0048	15	2.6×10^{-4}	yes	yes	yes
	1,000	66.5	66.6	0.9977	1.0056	15	2.6×10^{-4}	yes	yes	yes
`mixed-overlap`	10	53.0	52.8	1.0016	1.0047	14	2.1×10^{-3}	yes	yes	yes
	100	63.6	63.8	0.9988	1.0003	16	1.5×10^{-5}	yes	yes	yes
	1,000	76.8	76.9	1.0000	1.0043	16	1.5×10^{-5}	yes	yes	yes

at 1,048,576 steps and failed with the budget error at 4,194,304. Each then compiled at the budget found by halving the last factor of four, shown in Table 10. At the harness's budget, the compiles of those tables took 5,582 to 7,090 MiB of memory. These are compiler measurements of code that never runs, so they need no sanitizer record.

Table 10: The costs of the compile-time tables: clang on L at the native flags, and MSVC on W, one compile at a time. Clang's compile time and object size cover every table of a size. The steps and MSVC's columns cover the tables of table seed 111 at each size, and the GitHub table. Steps are the smallest budget of constant evaluation tried under which a table compiled. Peak memory is the compiler's peak committed memory at the harness's budget.

		Clang on L			**MSVC on W**		
m	**Tables**	**Compile (s)**	**Object (B)**	**Steps**	**Steps**	**Peak (MiB)**	**Object file (B)**
10	119	1.64 to 1.79	1,717 to 3,253	1,048,576	1,048,576	167 to 204	40,759 to 42,184
100	119	2.56 to 3.10	8,965 to 14,581	1,048,576 to 2,097,152	1,048,576	582 to 781	47,998 to 53,087
1,000	119	14.83 to 19.01	71,429 to 148,853	16,777,216	8,388,608	5,582 to 7,090	110,530 to 178,652
GitHub, 203	1	4.57	21,301	4,194,304	2,097,152	1,311	60,245

5.10. Exploratory Runs

The exploratory runs decide nothing and have no intervals (Table 11). On the GitHub table, v2 took 0.6125 of the time of the fastest competitor, `gin`, over 16 pairs. It ran 554.9 net instructions per lookup against 821.6. With half the queries missing, v2's ratio to the fastest competitor ranged from 0.1615, at `static` at $m = 100{,}000$, to 0.9498, at `mixed-disjoint` at $m = 10$. No cell was above one. v2 ran more net instructions than the fastest competitor in 4 of those cells, all at $m = 10$.

Long literals did not close the gap. Their ratios ran from 0.2373 to 0.4867, and v2 ran fewer net instructions in all 14 cells. The Zipf ring gave 0.1741 to 0.4898, again with fewer net instructions in every cell. Without decoys, gin holds the `mixed-overlap` tables, and v2's ratio ran from 0.3658 to 0.7816 up to $m = 10{,}000$. At $m = 100{,}000$ the analysis failed that cell. There gin and Boost.URL agreed in 13 of 16 processes and differed as declared in the others. An arm must agree in all of a cell's processes or in none.

Table 11: The exploratory runs, against the fastest competitor of each cell. R is the number of pairs per cell. "v2 / fastest" gives the range over cells of the median over pairs of the per-pair ratio. The last column counts the cells where v2 ran more net instructions than the fastest competitor.

Run	**Cells**	R	**v2 / fastest**	**More instructions**
GitHub table	one	16	0.6125	none
miss-heavy ring	35	3	0.1615 to 0.9498	4
long literals	14	3	0.2373 to 0.4867	0
Zipf ring	14	3	0.1741 to 0.4898	0
`mixed-overlap`, no decoys	4	16	0.3658 to 0.7816	0

The ablation arm shows what the rework of Section 3.7 changed. Against it, v2's lookup time ranged from 0.9842 to 1.0073 of the ablation arm's over the 35 cells, and its net instructions were equal (1.0000). The build took 0.3410 to 0.8360 of the ablation arm's time, and the table's heap bytes were 0.6450 to 1.0000 of its bytes. Appendix Appendix C gives every cell. The rework improved the build time and the heap, and left the lookup unchanged.

6. Discussion

6.1. What the Results Support

The results name which choices matter in the setting of Section 4.2, and where they lose. v2 was faster than the fastest competitor in 33 of 35 cells, and not slower in one more. A cell's fastest competitor is one that takes

part in that cell. The competitors left out are named with their reasons in Section 5.1, and the paper does not claim that they are slower.

The advantage grows with the table. v2's work per lookup barely depends on m. On `static` it ran 214.2 net instructions per lookup at $m = 10$ and 214.8 at $m = 100{,}000$, while the fastest competitor's count rose from 252.8 to 736.2. A walk over path segments takes one step per segment, so its length follows the depth of the route, not the number of routes. The width of a node matters only through its scan of up to 8 literal children, or one probe of their hashed slots. In every cell, v2 also took fewer branch misses than the median competitor.

6.2. Small Tables Against gin

Every loss or partial result of v2 in the lookup comparison sits at $m = 10$ and against gin. These are the 2 H2 losses, the one cell that does not pass H1, and the one cell that passes non-inferiority only. With ten routes, first words rarely repeat, so most nodes below the root have one child. A node of gin's tree compares its whole prefix at once, while v2 takes one step per segment. The development round attributed the instruction loss at `mixed-disjoint` to this difference. The rework tried to remove it with chains, and its rules rejected every attempt (Section 3.7).

An instruction count is not a time, however. At `param-last` with $m = 10$, v2 ran more net instructions than gin and was still faster, with the ratio 0.9053. Part of v2's count is the harness's adapter, which copies the captured values into the harness's own structure and folds them. gin folds its values inside Go, and its null pass does not. Appendix Appendix A lists these differences between the arms. We kept them, because removing them would change the harness for every arm.

At `wild` with $m = 10$, the development builds favoured v2. The run differs from those builds in the instruction set that every arm targets and in the build level of the Go arms. Both are candidate causes of the change in this cell, and neither was tested.

6.3. Build Time and the Compile-Time Table

The rework cut v2's build to 0.3410 to 0.8360 of the pre-release's time and reduced its heap. It was not enough in 4 cells, at `static` and `param-first` with $m = 10{,}000$ and 100,000. We did not profile what remains of the build there. A server builds its table once at start, and v2's slowest build of the largest tables took 156 ms. As pre-specified, H4's claim is not made.

The compile-time table passed its per-cell test in 20 of 21 cells. The rule needs all of them, so H7 does not hold. The one cell that fails has 4 pairs at or above the margin, where the rule allows 3, and its interval lies wholly above one. One of them, at table seed 124, ran more cycles with the same instructions. The rework had already given both tables one layout. What remains between them includes where the table lies, in the binary or on the heap, and where the code lies. The pre-specified run with a heap copy, made afterwards, found single slow pairs at other seeds and in each arm. Neither the table seed nor the table's place in memory explains the cell, and its cause stays open. Zero-overhead abstraction is therefore not established here for compile-time tables Stroustrup (2012).

The costs set a practical limit. A compile-time table of 1,000 routes took 14.83 to 19.01 s to compile with clang. With MSVC it took 5,582 to 7,090 MiB of memory, and MSVC crashed under the smallest budget of its search. A compile-time table is cheap at 10 and 100 routes and expensive at 1,000.

6.4. End to End

H6 compares v2 with v1 inside this paper's server. That comparison says nothing about gin, matchit or any other external router. Section 5.8 records that the external comparison is absent. The lookup difference against v1 reached the client in every cell of H6. In the minimal server the null router's time per request was about 8.16 μs, so routing is a visible share of a request in a server this lean. The measured ratio stayed within 0.9962 to 1.1110 of the ratio that the lookup times predict. The share would be smaller in a full framework with more work per request.

7. Threats to Validity

The threats below are the limits of the design in Section 4 and of the calls in Appendix Appendix A. They use no result beyond the runs already reported.

- **One processor model and one core.** Every time comes from one core of one Zen 3 processor. Another microarchitecture may rank close cells differently, above all the cells at $m = 10$ against gin.
- **One compiler per language.** C and C++ were compiled by clang only, Rust by rustc and Go by its own toolchain. GCC and MSVC builds were not timed. MSVC appears only in the costs of compile-time tables.
- **The native instruction set.** Every arm was built for L's own processor. A generic build would give other code and other times, which may matter most for the small tables against gin.
- **Loopback for H6.** The end-to-end run used loopback, one server core and one request in flight per connection. A network, more cores or pipelining would change the share of routing.
- **The minimal server.** It parses, routes and answers a fixed body. A full framework adds work per request, which dilutes the difference between routers.
- **External routers end to end.** The archived end-to-end run compares v2 with v1 in this paper's server. It records no request rate for gin, matchit or any other external router. Section 5.8 leaves that block out. A lookup result stays a statement about lookups.
- **Development on the same host.** The engineering of v2 ran on L, on other seeds. The run's seeds were held out, but the design choices may still suit L's microarchitecture better than others.
- **The Drogon transcription.** Drogon's router needs a running application, so its lookup was transcribed into the harness from its source. The file's hash and line ranges are pinned. A transcription can still differ from the router as Drogon runs it.
- **The gin shim.** gin's tree is vendored unchanged, but the parts of gin's engine that a lookup uses are reproduced in a small shim. They are the reset of the context, the method's tree and the search in gin's default configuration. chi's arm uses a similar shim of its multiplexer. A shim can differ from the engine it stands for.
- **The repeated ring.** Every timed pass repeats the same 4,096 queries. Predictors and caches learn the ring, and a large table's hot part is at most the ring's routes. Real traffic, with more distinct paths, may favour other structures. The Zipf and miss-heavy rings explore two departures only.
- **Different measurement boundaries.** C and C++ arms are called in process. The Rust arms are called through one C function per lookup. The Go arms run their timed loops inside Go, entered through cgo. Their nulls remove the loops, and other costs of the boundary remain. A ratio of a C++ time to a Go time crosses that boundary. Appendix Appendix A lists the calls.
- **The cut for slow arms.** A competitor whose passes ran on a prefix of the ring has optimistic times and counters. None was the fastest competitor of an H1 cell, but the cut competitors enter the median of H2's branch misses, which Table 5 gives with and without them.

8. Conclusions

The question is which design choices in route matching matter, and when. We surveyed the route matchers of sixteen routers in C, C++, Rust and Go, and reworked the author's RegexMatcher library into a route matcher, RegexMatcher v2. Its lookup walks a trie of path segments stored in flat arrays, answers literal-only methods from an exact-match table, and never allocates. Its front end checks patterns, handler signatures and duplicate routes when the program compiles.

Under hypotheses pre-specified before the run, and in the setting of Section 4.2, v2 was faster than the fastest competitor in 33 of 35 cells. It was not slower in one more. At the `wild` shape with 10 routes, neither claim holds against gin. v2 ran more instructions than gin in 2 cells of 10 routes. It built its table more slowly than the median competitor in 4 cells with large tables. The compile-time table passed its per-cell test in 20 of 21 cells; the rule needs all, so H7 does not hold. No lookup of v2 allocated, every probe of the path rules passed, and the compile-time checks rejected every malformed case. In a minimal server, the lookup difference against v1 reached the client in every cell.

Four questions remain open. The first is whether the results hold on other processors and with other compilers, above all for small tables. The second is why the compile-time table was slower at `param-last` at $m = 1{,}000$. The pre-specified run with a heap copy did not tie the slow pairs to the table's place in memory, and it cannot rule out the placement of code. The third is what remains of the build time for large literal tables. The fourth is the end-to-end latency against the external routers. This manuscript leaves that comparison unmeasured.

Appendix A. What Each Arm's Timed Lookup Includes

For every arm, the timed loop calls the arm's own pass if it has one, as the Go arms do. Otherwise it runs the harness's loop over the compact copy of the ring. The loop adds each route number, and each captured value's length and first byte, to a sink that the compiler cannot remove. For an arm without a 405 of its own, a "no route" answer starts a lookup of the same path under every other method that has routes. Any hit makes the answer 405. On the all-hit rings of the grid, that fallback runs only after a wrong answer, but its test runs on every lookup. The allocation pass and the latency samples call each lookup through the harness, so a Go arm's latency includes one call across cgo.

"Index" in Table A.12 means that the harness passes the method as a number and the adapter indexes one router per method. "String" means that the router takes the method's name.

What remains unequal is stated rather than removed, because removing it would leave the router's interface:

- The copies of captured values that a router's interface makes are in its time. This holds for Crow, Oat++'s tail, Drogon, Pistache, glaze and cpp-httplib's named values. v2 returns views.
- The adapters of Oat++ and Pistache index one router per method, where the framework looks the method up in a map. This removes a step from those arms and never adds one. Boost.URL's example router, matchit and path-tree have no methods, so their adapters keep one router per method.
- gin's adapter copies the values of a query that needed a return. Only `mixed-overlap` has such queries, and there gin is incomparable.
- v2's lookup is an out-of-line call, while a header-only competitor can be inlined into the timed loop. This adds work to v2 only.
- The Go arms take the method as a string, inside the timed lookup.
- r3 creates and frees one match entry per lookup, as its interface requires.

Table A.12: The timed call of each arm. "Loop" is the loop that times the arm, and "null" the null that H2 subtracts.

Arm	Timed call	Method	Captured values	405	Loop; null
`regexmatcher-v2`	`find_v2`, out of line, which calls `matcher::route::find`	index	views into the path	its own	harness; `null`
`regexmatcher-v2-ct`	the same, on the compile-time table	index	views	its own	harness; `null`
`regexmatcher-v2-r1`	the pre-release's lookup, out of line	index	views	its own	harness; `null`
`regexmatcher-v1`	the wrapper's match, which calls v1's matching with groups	index	offsets; a new result vector per call	harness	harness; `null-no405`
radix, hash, regex	the harness's lookups	index	views (hash: none)	harness	harness; `null-no405`
Crow	`crow::Trie::find`	index	copied into strings; views of them	harness	harness; `null-no405`
Oat++	`Router::getRoute`	index	views; the tail copied into a string	harness	harness; `null-no405`
Boost.URL	`parse_path`, then the router's `find`	index	views	harness	harness; `null-no405`
r3	one match entry created, matched and freed per lookup	a bit in the entry	views	harness	harness; `null-no405`
Drogon	the transcribed lookup: a lower-cased copy, two maps, then a regular expression per route	index	a new vector of strings per match	harness	harness; `null-no405`
Pistache	`sanitizeResource`, then `findRoute`	index	copies, its only public read	harness	harness; `null-no405`
matchit	`Router::at` behind one C call	index	offsets	harness	harness; `null-no405`
actix-router	`Path::set`, then `Router::recognize`, behind one C call	index	offsets, not decoded	harness	harness; `null-no405`
path-tree	`PathTree::find` behind one C call	index	offsets	harness	harness; `null-no405`
uWebSockets	`HttpRouter::route`; the handler stores the route	string	views; none for a catch-all	harness	harness; `null-no405`
glaze	`route_table::match`	a switch	decoded strings in a map; one find per name	harness	harness; `null-no405`
cpp-httplib	the request's path assigned, then the method's matchers in order	index	named values copied; groups as views	harness	harness; `null-no405`
httprouter	`Router.Lookup`, then the handler, in Go	string	offsets	the harness's rule, in Go	its own; its own
gin	the vendored tree's search, then the handlers, in Go	string	offsets; values after a return copied	the harness's rule, in Go	its own; its own
net/http	`ServeMux.ServeHTTP` on one reused request, in Go	string	the handler copies each value	its own	its own; its own
chi	the context reset, the method map, the tree search, then the handler, in Go	string	offsets	the harness's rule, in Go	its own; its own

Appendix B. Competitor Configurations

The C and C++ arms are built by clang 22.1.8 in CMake's Release configuration, without link-time optimization. Every target gets the same flags, `-O3 -DNDEBUG -std=c++23 -march=native`. The libraries that the harness fetches and builds get the same flags. The Rust arms use `-C target-cpu=native` and the harness's release profile, with the prebuilt standard library. The Go arms use `GOAMD64=v3`, the highest level that L supports, and the C that cgo compiles gets the native flag too. Table B.13 gives what each competitor adds and the documentation it follows. "Library default" means that we found no documented advice.

Table B.13: Each competitor's configuration and its source.

Competitor	Build and run-time configuration	The library's documentation
Crow v1.3.4	header only, standalone Asio; log level Warning	no optimization advice; its setup page compiles without flags; its logging guide sets Warning
Oat++ 1.3.1	its own CMake, static; its object counters disabled	its build file says the option disables object counting for release builds, for performance
Boost.URL 1.92.0	the library and its example router, static	library default
r3, 2.0 branch, 83d362f	its sources compiled by the harness; PCRE2 10.49 by its CMake, static, without JIT	no build flags in its README; r3 never compiles a pattern with PCRE2's JIT, so the JIT would not apply
Drogon v1.9.13	its lookup transcribed into the adapter, common flags; the file's hash and line ranges pinned	its guide asks for a Release build of the library, which the harness does not build
Pistache v0.4.26	its own CMake, Release, static, tests and TLS off	its README uses meson's release build; CMake's Release is used here
uWebSockets v20.80.0	header only; only the router compiled; native flags	no flags in its README; its own example build uses a native build, `-O3` and link-time optimization
glaze v9.0.0	header only; its CMake adds nothing else on Linux; native flags	its performance guide, written for its serialization, recommends `-O2` or `-O3` and a native build when the program runs where it was built
cpp-httplib v0.58.0	header only, no TLS, no compression	library default
matchit 0.9.2	common Rust	library default
actix-router 0.5.4	common Rust, default features	library default
path-tree 0.8.3	common Rust	library default
httprouter v1.3.0	common Go	library default
gin v1.12.0	common Go; the tree vendored and called without gin's engine	its deployment page sets the release mode, which governs debug output only; the vendored lookup reads no mode
net/http, Go 1.27.1	common Go	no build advice
chi v5.3.2	common Go; its sources vendored, each with its hash	library default

Link-time optimization stays off for every C and C++ arm, v2 included. Of the documentation read, only uWebSockets' example build uses it.

Appendix C. The Ablation Arm per Cell

Supplementary material

The following supporting information can be downloaded at `https://github.com/Alex-Tsvetanov/paper-typed-routing`: Table S1, every arm in every cell of the main grid. Table S1 gives each value as the median over the arm's processes. It is the file `results/round2/all_arms.csv` of the paper's repository. Its columns are the arm's role, its processes and their statuses, and whether it takes part in the cell, with the reason when it does not. They also hold the time per lookup, the raw and net instructions, the branch misses, the cycles, and the L1d, L2 and DTLB misses. The allocations per lookup, the build time, the bytes allocated by the build and the heap bytes of the table follow. The latency percentiles and, for the Go arms, the cost of an empty call across cgo complete them.

CRediT authorship contribution statement

Alex I. Tsvetanov: conceptualization, methodology, software, validation, formal analysis, investigation and resources. The same author did the data curation, the writing of the original draft, the review and editing, and the visualization. The same author did the supervision, the project administration and the funding acquisition.

Table C.14: v2 against the ablation arm, which keeps the pre-release's build and layout. Each value is the median over the 16 pairs of v2's value over the ablation arm's. The run decides nothing from these ratios.

Shape	m	**Time**	**Net instructions**	**Build time**	**Heap bytes**
`static`	10	0.9942	1.0000	0.7939	1.0000
	100	0.9975	1.0000	0.7670	0.9426
	1,000	0.9980	1.0000	0.3934	0.8976
	10,000	0.9978	1.0000	0.5563	0.9429
	100,000	0.9978	1.0000	0.6898	0.9740
`param-last`	10	0.9975	1.0000	0.7943	0.8510
	100	0.9993	1.0000	0.7890	0.8510
	1,000	0.9937	1.0000	0.3661	0.8612
	10,000	1.0029	1.0000	0.6252	0.8333
	100,000	1.0054	1.0000	0.7511	0.8818
`param-first`	10	1.0004	1.0000	0.7923	0.7697
	100	1.0007	1.0000	0.7790	0.6991
	1,000	1.0004	1.0000	0.4042	0.7101
	10,000	1.0009	1.0000	0.5565	0.8902
	100,000	0.9923	1.0000	0.7442	0.9115
`rest`	10	0.9997	1.0000	0.8344	0.7221
	100	0.9986	1.0000	0.8069	0.6450
	1,000	0.9981	1.0000	0.4539	0.6613
	10,000	0.9971	1.0000	0.6158	0.8337
	100,000	1.0046	1.0000	0.7595	0.8818
`wild`	10	1.0073	1.0000	0.8027	0.8510
	100	0.9997	1.0000	0.7992	0.8510
	1,000	0.9985	1.0000	0.3459	0.8612
	10,000	1.0002	1.0000	0.6285	0.8333
	100,000	0.9867	1.0000	0.7755	0.8818
`mixed-disjoint`	10	0.9992	1.0000	0.8254	0.8506
	100	0.9986	1.0000	0.8094	0.7578
	1,000	0.9937	1.0000	0.3541	0.7760
	10,000	0.9965	1.0000	0.5358	0.7775
	100,000	0.9842	1.0000	0.6572	0.6482
`mixed-overlap`	10	0.9978	1.0000	0.8360	0.8758
	100	0.9997	1.0000	0.7881	0.7590
	1,000	0.9971	1.0000	0.3410	0.8263
	10,000	0.9987	1.0000	0.5309	0.9525
	100,000	0.9887	1.0000	0.6020	0.7093

Funding

The publication of this paper was financially supported by the scientific research sector of the Technical University of Sofia.

Declaration of competing interest

The author declares no conflicts of interest. The funder had no role in the design of the study or in the collection, analysis or interpretation of data. The funder had no role in the writing of the manuscript or in the decision to publish the results.

Data availability

The paper's repository, `https://github.com/Alex-Tsvetanov/paper-typed-routing`, holds the harness and the minimal server at commit `f626955ebd94398a1376af4d48d61ee7a9ca9f6c`, with the analysis, the pre-specification and the derived tables. It also holds the sources of the T1 tools that ran H6 (`lab/t1/`). The raw runs are the assets of its release `data-2026-10` (`https://github.com/Alex-Tsvetanov/paper-typed-routing/releases/tag/data-2026-10`), and `results/round2/archives.sha256` lists them. The raw main grid is `p2-raw-2026-09-30-L-publication.tar.gz`, SHA-256 `efc3b2fc6c74bc21d8ac7a152abfb6704345b309352b2971d219c4612601e8a6`. The repeated check of its answers is `p2-raw-2026-09-30-L-publication-verify-repeat.tar.gz`, SHA-256 `19600f543dfd56d3bc548d959d550fbc136ec6262088abc223246b2c01ff4420`. The clang costs of the compile-time tables are `p2-raw-2026-09-30-L-publication-ct.tar.gz`, SHA-256 `d88b08c2de346e8defd136c7c317baf9ed4eadf736deffeb2d10f53e04f61744`. The MSVC costs are `p2-raw-2026-10-01-W-ct-msvc.tar.gz`, SHA-256 `1e32cfbffc60a9f099cf026c32bd23681e57fb220ce261334fc75e404ab5d06a`. The end-to-end run is `p2-raw-2026-09-30-L-h6.tar.gz`, SHA-256 `effe075b696c916ac83391fa58d627d5e8729c874a2be693b1571e28f7c90da7`. The exploratory runs are `p2-raw-2026-09-30-L-exploratory-github.tar.gz`, SHA-256 `7c3b05894b8f9a4408708b8ee3dd259e1e6e2bf6c36a570f2ef46ae0ec0a24b8`, and `p2-raw-2026-09-30-L-exploratory-miss.tar.gz`, SHA-256 `70216a094cf9b752144bd29e35c1fdbf78e702d9ed05483afe41c1c7cf4a4fd8`. They are also `p2-raw-2026-09-30-L-exploratory-vocab-long.tar.gz`, SHA-256 `e72917dc66b7d8a27c194ec60ea7337921459ae06068197fbbab7e893a6dac1e`; `p2-raw-2026-09-30-L-exploratory-zipf.tar.gz`, SHA-256 `57b4d973afb050bb2d8794ef45e5fcf3ea30698ca6f36b0c2312ff2df525cef7`; and `p2-raw-2026-09-30-L-exploratory-no-decoy.tar.gz`, SHA-256 `0a57f2134cf2be69157194c60410885a16d45a94db10587c05356e3f81f916f8`. The heap-block follow-up of H7 is `p2-raw-2026-10-02-L-exploratory-ct-heap.tar.gz`, SHA-256 `ee2571a8f29ea55e549fe647726933122194d2050219b0823daf18313fc72912`. The sanitizer records that gated the harness and the server, those made later for the published commit, and those behind H5(a) and H5(c) are in `lab/sanitizer-records/` of the repository. Each record names the SHA-256 of an archive of its logs, which are not published. RegexMatcher v2 is commit `d1d73e99b7b26f54ba75354f4880780ece32e8ed`. It is available at `https://github.com/cpp-for-everything/RegexMatcher` in release 2.1.0.0. [For Alex: the DOIs of the paper's repository and of its release `data-2026-10`, which need Alex's account (for example a Zenodo archive of the release).]

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the author used Cursor agents to draft and revise the manuscript. After using these tools, the author reviewed and edited the content as needed and takes full responsibility for the content of the article.

Vitae

Alex I. Tsvetanov is with the Faculty of Computer Systems and Technologies at the Technical University of Sofia. He wrote this measurement of the public RegexMatcher library.

Acknowledgments

The author would like to thank the scientific research sector of the Technical University of Sofia for financial support for publishing the present paper.

References

Actix contributors, 2026. actix-router. URL: https://github.com/actix/actix-web/tree/router-v0.5.4/actix-router. crate version 0.5.4, tag router-v0.5.4 of actix-web, commit 6e4e040ee033f330f736dbfe9c4a0c121f8ca94a, February 2026.

Ahmed, I., 2026. matchit: A high performance, zero-copy URL router. URL: https://github.com/ibraheemdev/matchit. crate version 0.9.2, tag v0.9.2, commit 4b5a2c356022f544187d6e67325d8e98dd7f11ca, April 2026.

Amsterdam, J., 2024. Routing enhancements for Go 1.22. URL: https://go.dev/blog/routing-enhancements. the Go Blog, February 2024.

Andersson, A., Nilsson, S., 1993. Improved behaviour of tries by adaptive branching. Information Processing Letters 46, 295-300. doi:10.1016/0020-0190(93)90068-K.

Aoe, J.I., 1989. An efficient digital search algorithm by using a double-array structure. IEEE Transactions on Software Engineering 15, 1066-1077. doi:10.1109/32.31365.

Asai, H., Ohara, Y., 2015. Poptrie: A compressed trie with population count for fast and scalable software IP routing table lookup, in: 2015 ACM Conference on Special Interest Group on Data Communication (SIGCOMM), ACM. pp. 57-70. doi:10.1145/2785956.2787474.

Balat, V., 2006. Ocsigen: Typing web interaction with objective Caml, in: 2006 Workshop on ML, ACM, Portland, OR, USA. pp. 84-94. doi:10.1145/1159876.1159889.

Belazzougui, D., Botelho, F.C., Dietzfelbinger, M., 2009. Hash, displace, and compress, in: Algorithms, ESA 2009. Springer, Berlin, Heidelberg. Lecture Notes in Computer Science, pp. 682-693. doi:10.1007/978-3-642-04128-0_61.

Berners-Lee, T., Fielding, R., Masinter, L., 2005. Uniform Resource Identifier (URI): Generic Syntax. RFC 3986. RFC Editor. doi:10.17487/RFC3986.

Blanvillain, O., 2022. Type-safe regular expressions, in: 13th ACM SIGPLAN Scala Symposium, ACM, Berlin, Germany. pp. 1-8. doi:10.1145/3550198.3550425.

Boost.URL contributors, 2026. Boost.URL, example/router. URL: https://github.com/boostorg/url/tree/boost-1.92.0/example/router. tag boost-1.92.0, commit a34f3a4a1a0e695f679453af71dfc050a59170e6, May 2026.

Browne, S., Dongarra, J., Garner, N., Ho, G., Mucci, P., 2000. A portable programming interface for performance evaluation on modern processors. The International Journal of High Performance Computing Applications 14, 189-204. doi:10.1177/109434200001400303.

Celis, P., Larson, P.A., Munro, J.I., 1985. Robin hood hashing, in: 26th Annual Symposium on Foundations of Computer Science (SFCS 1985), IEEE, Portland, OR, USA. pp. 281-288. doi:10.1109/SFCS.1985.48.

Chlipala, A., 2015. Ur/Web: A simple model for programming the web, in: 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL), ACM, Mumbai, India. pp. 153-165. doi:10.1145/2676726.2677004.

cpp-httplib contributors, 2026. cpp-httplib. URL: https://github.com/yhirose/cpp-httplib. version v0.58.0, commit 4f3f9ef19be83ae97a5d9a059432dc00e445b7ab, September 2026.

CrowCpp contributors, 2026a. Crow. URL: https://github.com/CrowCpp/Crow. version v1.3.4, commit ae0fef0ee67eec897e401321b99b6dd7cfbdc155, September 2026.

CrowCpp contributors, 2026b. Crow: CROW_ROUTE and its handler call. URL: https://github.com/CrowCpp/Crow/tree/ae0fef0ee67eec897e401321b99b6dd7cfbdc155/include/crow. version v1.3.4, commit ae0fef0ee; files include/crow/app.h, utility.h, routing.h and middleware.h, September 2026.

Curtsinger, C., Berger, E.D., 2013. STABILIZER: Statistically sound performance evaluation, in: Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), ACM, Houston, TX, USA. pp. 219-228. doi:10.1145/2451116.2451141.

Czech, Z.J., Havas, G., Majewski, B.S., 1992. An optimal algorithm for generating minimal perfect hash functions. Information Processing Letters 43, 257-264. doi:10.1016/0020-0190(92)90220-P.

Danvy, O., 1998. Functional unparsing. Journal of Functional Programming 8, 621-625. doi:10.1017/S0956796898003104.

De La Briandais, R., 1959. File searching using variable length keys, in: Papers Presented at the March 3-5, 1959, Western Joint Computer Conference (IRE-AIEE-ACM '59 Western), ACM, San Francisco, CA, USA. pp. 295-298. doi:10.1145/1457838.1457895.

Degermark, M., Brodnik, A., Carlsson, S., Pink, S., 1997. Small forwarding tables for fast routing lookups, in: ACM SIGCOMM '97 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communication, ACM, Cannes, France. pp. 3-14. doi:10.1145/263105.263133.

DiCiccio, T.J., Efron, B., 1996. Bootstrap confidence intervals. Statistical Science 11, 189-228. doi:10.1214/ss/1032280214.

Dos Reis, G., Stroustrup, B., 2010. General constant expressions for system programming languages, in: 2010 ACM Symposium on Applied Computing (SAC), ACM, Sierre, Switzerland. pp. 2131-2136. doi:10.1145/1774088.1774537.

Drogon contributors, 2026. Drogon. URL: https://github.com/drogonframework/drogon. version v1.9.13, commit 4c5430757ea5451a7c38fbbef4b4bef7dbb47f2f, May 2026.

Dsouza, I., Dewan, H., Singh, P., Hansdah, R., 2018. [WiP] HTU: An efficient hash-trie data structure for URL matching for programmable web resources, in: 2018 IEEE 11th Conference on Service-Oriented Computing and Applications (SOCA), IEEE, Paris, France. pp. 233-238. doi:10.1109/SOCA.2018.00041.

Dusíková, H., 2019. Compile time regular expressions. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1433r0.pdf. wG21 paper P1433R0, ISO/IEC JTC1/SC22/WG21, January 2019.

Dusíková, H., 2026. Compile time regular expressions (CTRE) v3. URL: https://github.com/hanickadot/compile-time-regular-expressions. version v3.11.0, commit 4ab18abfd88ccfc867734bf5d31b13285854577e, May 2026.

Eatherton, W., Varghese, G., Dittia, Z., 2004. Tree bitmap: Hardware/software IP lookups with incremental updates. ACM SIGCOMM Computer Communication Review 34, 97-122. doi:10.1145/997150.997160.

Efron, B., 1987. Better bootstrap confidence intervals. Journal of the American Statistical Association 82, 171-185. doi:10.1080/01621459.1987.10478410.

Efron, B., Tibshirani, R.J., 1993. An Introduction to the Bootstrap. Springer US. doi:10.1007/978-1-4899-4541-9.

Fielding, R., Nottingham, M., Reschke, J. (Eds.), 2022. HTTP Semantics. Number 9110 in RFC, RFC Editor. doi:10.17487/RFC9110.

Fredkin, E., 1960. Trie memory. Communications of the ACM 3, 490-499. doi:10.1145/367390.367400.

Fredman, M.L., Komlós, J., Szemerédi, E., 1984. Storing a sparse table with $O(1)$ worst case access time. Journal of the ACM 31, 538-544. doi:10.1145/828.1884.

Free Software Foundation, 2025. GNU gperf: A perfect hash function generator. URL: https://www.gnu.org/software/gperf/. version 3.3, April 2025.

Georges, A., Buytaert, D., Eeckhout, L., 2007. Statistically rigorous Java performance evaluation, in: 22nd Annual ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA), ACM, Montreal, QC, Canada. pp. 57-76. doi:10.1145/1297027.1297033.

Gin contributors, 2026. Gin. URL: https://github.com/gin-gonic/gin. version v1.12.0, commit 73726dc606796a025971fe451f0aa6f1b9b847f6, February 2026.

Glaze contributors, 2026. Glaze. URL: https://github.com/stephenberry/glaze. version v9.0.0, commit d78832c82289c61a9315bfbc35332cec9f4e93ca, September 2026; the router is include/glaze/net/http_router.hpp.

go-chi contributors, 2026. chi. URL: https://github.com/go-chi/chi. version v5.3.2, commit 38939062c5df4d3e8814aad1a488983112627ced, August 2026.

Gupta, P., McKeown, N., 2001. Algorithms for packet classification. IEEE Network 15, 24-32. doi:10.1109/65.912717.

Heinz, S., Zobel, J., Williams, H.E., 2002. Burst tries: A fast, efficient data structure for string keys. ACM Transactions on Information Systems 20, 192-223. doi:10.1145/506309.506312.

Hoefler, T., Belli, R., 2015. Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results, in: International Conference for High Performance Computing, Networking, Storage and Analysis (SC15), ACM, Austin, TX, USA. pp. 1-12. doi:10.1145/2807591.2807644.

Holm, S., 1979. A simple sequentially rejective multiple test procedure. Scandinavian Journal of Statistics 6, 65-70.

Kalibera, T., Jones, R., 2013. Rigorous benchmarking in reasonable time, in: 2013 International Symposium on Memory Management (ISMM), ACM, Seattle, WA, USA. pp. 63-74. doi:10.1145/2464157.2464160.

Laaber, C., Scheuner, J., Leitner, P., 2019. Software microbenchmarking in the cloud. how bad is it really? Empirical Software Engineering 24, 2469-2508. doi:10.1007/s10664-019-09681-1.

Leis, V., Kemper, A., Neumann, T., 2013. The adaptive radix tree: ARTful indexing for main-memory databases, in: 2013 IEEE 29th International Conference on Data Engineering (ICDE), IEEE, Brisbane, QLD, Australia. pp. 38-49. doi:10.1109/ICDE.2013.6544812.

Leitner-Ankerl, M., 2026. nanobench. URL: https://github.com/martinus/nanobench. version v4.6.0, commit b709728dba4df0356d467abfb0ba20781a8031c0, August 2026.

Linux man-pages project, 2026. perf_event_open(2): Set up performance monitoring. URL: https://man7.org/linux/man-pages/man2/perf_event_open.2.html. linux man-pages 6.19, May 2026.

Mainland, G., 2007. Why it's nice to be quoted: Quasiquoting for Haskell, in: ACM SIGPLAN Workshop on Haskell, ACM, Freiburg, Germany. pp. 73-82. doi:10.1145/1291201.1291211.

Mestanogullari, A., Hahn, S., Arni, J.K., Löh, A., 2015. Type-level web APIs with Servant: An exercise in domain-specific generic programming, in: 11th ACM SIGPLAN Workshop on Generic Programming (WGP), ACM, Vancouver, BC, Canada. pp. 1-12. doi:10.1145/2808098.2808099.

Microsoft, 2026. /constexpr (control constexpr evaluation). URL: https://learn.microsoft.com/en-us/cpp/build/reference/constexpr-control-constexpr-evaluation?view=msvc-170. microsoft C++ documentation, Microsoft Learn.

Morrison, D.R., 1968. PATRICIA: Practical algorithm to retrieve information coded in alphanumeric. Journal of the ACM 15, 514-534. doi:10.1145/321479.321481.

Mytkowicz, T., Diwan, A., Hauswirth, M., Sweeney, P.F., 2009. Producing wrong data without doing anything obviously wrong!, in: 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), ACM, Washington, DC, USA. pp. 265-276. doi:10.1145/1508244.1508275.

Nilsson, S., Karlsson, G., 1999. IP-address lookup using LC-tries. IEEE Journal on Selected Areas in Communications 17, 1083-1092. doi:10.1109/49.772439.

Oat++ contributors, 2024. Oat++. URL: https://github.com/oatpp/oatpp. version 1.3.1, commit 17ef2a7f6c8a932498799b2a5ae5aab2869975c7, February 2024.

Pagh, R., Rodler, F.F., 2004. Cuckoo hashing. Journal of Algorithms 51, 122-144. doi:10.1016/j.jalgor.2003.12.002.

Peterson, W.W., 1957. Addressing for random-access storage. IBM Journal of Research and Development 1, 130-146. doi:10.1147/rd.12.0130.

Pibiri, G.E., Trani, R., 2021. PTHash: Revisiting FCH minimal perfect hashing, in: 44th International ACM SIGIR Conference on Research and Development in Information Retrieval, ACM. pp. 1339-1348. doi:10.1145/3404835.3462849.

Pistache contributors, 2024. Pistache. URL: https://github.com/pistacheio/pistache. version v0.4.26, commit ddffda861aa49012dcda28f1362d0339e718cd52, December 2024.

Play Framework, 2026. HTTP routing, scala. URL: https://www.playframework.com/documentation/3.0.x/ScalaRouting. play Framework documentation, version 3.0.x.

Porkoláb, Z., Sinkovics, Á., 2010. Domain-specific language integration with compile-time parser generator library, in: Ninth International Conference on Generative Programming and Component Engineering (GPCE), ACM, Eindhoven, The Netherlands. pp. 137-146. doi:10.1145/1868294.1868315.

r3 contributors, 2026. libr3: A high-performance path dispatching library. URL: https://github.com/c9s/r3. branch 2.0, commit 83d362f80eeae0a5d5064f9db7ae6fd9ad27ea4e, September 2026.

Rocket contributors, 2024. Rocket. URL: https://github.com/rwf2/Rocket. version v0.5.1, commit f53611f74f2c777b9d7ebb40f8ec3f8dadd0a6f6, May 2024.

Ruiz-Sanchez, M.A., Biersack, E.W., Dabbous, W., 2001. Survey and taxonomy of IP address lookup algorithms. IEEE Network 15, 8-23. doi:10.1109/65.912716.

Schmidt, J., 2019. HttpRouter. URL: https://github.com/julienschmidt/httprouter. version v1.3.0, commit 4eec211fa4e8df74ed978dc5681612131854144f, September 2019.

Schmidt, J., 2020. Go HTTP router benchmark (go-http-routing-benchmark). URL: https://github.com/julienschmidt/go-http-routing-benchmark. commit d8f3b858995830fc29e26206d7d4d97ece34528c, July 2020.

Schuirmann, D.J., 1987. A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability. Journal of Pharmacokinetics and Biopharmaceutics 15, 657-680. doi:10.1007/BF01068419.

Serge sans Paille, 2024. frozen: A header-only, constexpr alternative to gperf for C++14 users. URL: https://github.com/serge-sans-paille/frozen. version 1.2.0, commit 06bee5321cecd9a9ffaceb164b5adb2ffafae781, June 2024.

Smith, R., Koeppe, T., Maurer, J., Perchik, D., 2020. Working draft, standard for programming language C++. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/n4861.pdf. wG21 document N4861, ISO/IEC JTC1/SC22/WG21, April 2020.

Snyder, J., Dionne, L., 2018. Class types in non-type template parameters. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0732r2.pdf. wG21 paper P0732R2, ISO/IEC JTC1/SC22/WG21, June 2018.

So, W., Narayanan, A., Oran, D., 2013. Named data networking on a router: Fast and DoS-resistant forwarding with hash tables, in: Architectures for Networking and Communications Systems (ANCS), IEEE. pp. 215-225. doi:10.1109/ANCS.2013.6665203.

Srinivasan, V., Varghese, G., 1999. Fast address lookups using controlled prefix expansion. ACM Transactions on Computer Systems 17, 1-40. doi:10.1145/296502.296503.

Stankov, I., Tsvetanov, A., 2024. Matching text from start to finish against multiple regular expressions, in: 2024 32nd National Conference with International Participation (TELECOM), IEEE, Sofia, Bulgaria. pp. 1-4. doi:10.1109/TELECOM63374.2024.10812214.

Stankov, I., Tsvetanov, A., 2026. RegexMatcher: Matching text against multiple regular expressions. URL: https://github.com/cpp-for-everything/RegexMatcher. tag 2.0.0.1, commit d16f30a81158d620e5a5514087b175a2251e4fa3, August 2026.

Stiffstream, 2025. RESTinio: easy_parser_router. URL: https://github.com/Stiffstream/restinio. version v.0.7.4, commit 0b7a523cb, file dev/restinio/router/easy_parser_router.hpp, February 2025.

Stroustrup, B., 2012. Foundations of C++, in: Seidl, H. (Ed.), Programming Languages and Systems (ESOP 2012). Springer, Berlin, Heidelberg. volume 7211 of *Lecture Notes in Computer Science*, pp. 1-25. doi:10.1007/978-3-642-28869-2_1.

Stroustrup, B., 2020. Thriving in a crowded and changing world: C++ 2006-2020. Proceedings of the ACM on Programming Languages 4, 1-168. doi:10.1145/3386320.

SwornSystems contributors, 2026. wayfind. URL: https://github.com/SwornSystems/wayfind. commit 2300ca7687f9f690ae8246dbedf3dfa1eb0eed75, August 2026; its benchmarks of Rust routers are in benches/.

Taylor, D.E., 2005. Survey and taxonomy of packet classification techniques. ACM Computing Surveys 37, 238-275. doi:10.1145/1108956.1108958.

The Clang Team, 2026. Clang compiler user's manual: -fconstexpr-steps. URL: https://clang.llvm.org/docs/UsersManual.html. clang documentation.

The Go Authors, 2026. The Go programming language, including net/http ServeMux. URL: https://github.com/golang/go. release go1.27.1, commit 862c888e612ac346c7c4d99c9392bdfd265f33b0, September 2026.

Tokio contributors, 2026. axum. URL: https://github.com/tokio-rs/axum. version 0.8.9, tag axum-v0.8.9, April 2026; its axum/Cargo.toml pins matchit 0.8.4.

uNetworking contributors, 2026. uWebSockets. URL: https://github.com/uNetworking/uWebSockets. version v20.80.0, commit 3ffd6f44c9c3c92c96345d9f96bd01ba9c025ab5, September 2026; the router is src/HttpRouter.h.

Virtanen, P., Gommers, R., Oliphant, T.E., Haberland, M., et al., 2020. SciPy 1.0: Fundamental algorithms for scientific computing in Python. Nature Methods 17, 261-272. doi:10.1038/s41592-019-0686-2.

viz-rs contributors, 2025. path-tree. URL: https://github.com/viz-rs/path-tree. crate version 0.8.3, tag v0.8.3, commit 158f3507c3ee64589f19645e9d0664b362f56cb1, March 2025.

vm-001, 2024. gateways-routing-benchmark. URL: https://github.com/vm-001/gateways-routing-benchmark. commit 8122a5cf2ed2cc2fb69f7bc7ab09022322a4b525, April 2024.

Waldvogel, M., Varghese, G., Turner, J., Plattner, B., 1997. Scalable high speed IP routing lookups, in: ACM SIGCOMM '97 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communication, ACM, Cannes, France. pp. 25-36. doi:10.1145/263105.263136.

Wang, Y., He, K., Dai, H., Meng, W., Jiang, J., Liu, B., Chen, Y., 2012. Scalable name lookup in NDN using effective name component encoding, in: 2012 IEEE 32nd International Conference on Distributed Computing Systems (ICDCS), IEEE. pp. 688-697. doi:10.1109/ICDCS.2012.35.

Weaver, V.M., Terpstra, D., Moore, S., 2013. Non-determinism and overcount on modern hardware performance counter implementations, in: 2013 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS), IEEE, Austin, TX, USA. pp. 215-224. doi:10.1109/ISPASS.2013.6557172.

Wellek, S., 2010. Testing Statistical Hypotheses of Equivalence and Noninferiority. Chapman and Hall/CRC. doi:10.1201/EBK1439808184.

Yesod contributors, 2025. yesod-core: Yesod.Routes.Parse. URL: https://github.com/yesodweb/yesod/blob/f0e2f04b7ddb93a92272a7a810a673bc9017b5a9/yesod-core/src/Yesod/Routes/Parse.hs. version 1.6.28.1, tag yesod-core-1.6.28.1, commit f0e2f04b7, October 2025.

Yesod Web Framework, 2026. Routing and handlers. URL: https://www.yesodweb.com/book/routing-and-handlers. yesod Web Framework Book, Version 1.6.

Yuan, H., Crowley, P., 2015. Reliably scalable name prefix lookup, in: 2015 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS), IEEE. pp. 111-121. doi:10.1109/ANCS.2015.7110125.

Zeng, P., Tan, Q., Meng, X., Shao, Z., Xie, Q., Yan, Y., Cao, W., Xu, J., 2017. A multi-pattern hash-binary hybrid algorithm for URL matching in the HTTP protocol. PLOS ONE 12, e0175500. doi:10.1371/journal.pone.0175500.

Zhang, Y., Xu, L., 2015. An algorithm for URL routing based on trie structure, in: 2015 12th Web Information System and Application Conference (WISA), IEEE, Jinan, China. pp. 157-160. doi:10.1109/WISA.2015.62.

Zverovich, V., 2021. std::format improvements. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2216r3.html. wG21 paper P2216R3, ISO/IEC JTC1/SC22/WG21, February 2021.

Zverovich, V., et al., 2026. fmt: A modern formatting library. URL: https://github.com/fmtlib/fmt. version 12.2.0, commit 1be298e1bd68957e4cd352e1f676f00e07dcfb57, June 2026.